

AI Hallucination Attack Surface: A Risk Assessment of Fake APIs and Libraries in AI-Generated Code

Submitted by
Hirushan Subasinghe
IM/2021/101

Supervised by
Mr. Dinesh Asanka

A dissertation submitted in partial fulfilment of the requirements
for the Degree of Bachelor of Science in Management and
Information Technology

**Department of Industrial Management
Faculty of Science
University of Kelaniya,
Sri Lanka
July 2026**

Declaration

I hereby state that this research report and findings presented in it are my own and it has not been submitted before nor is it currently being submitted for any other academic program. Where material has been used from other sources due recognition has been given by mentioning the source.

Name of the student: Hirushan Subasinghe

Student No: IM/2021/101

Signature:

Date: 17/07/2026

Abstract

The integration of Large Language Models (LLMs) into software engineering has significantly accelerated development, but it has also introduced a novel and critical supply-chain vulnerability: package hallucination. During code generation, LLMs frequently produce plausible but non-existent software dependencies and APIs, driven by training data limitations and the probabilistic nature of inference. This phenomenon creates a highly predictable attack surface that is exploited through slopsquatting, a threat in which adversaries pre-emptively register hallucinated package names, embedded with malicious payloads, on public registries such as npm or PyPI. This risk is substantially amplified by the rapid adoption of autonomous agentic workflows ("vibe coding"), in which AI agents function as confused deputies capable of automatically installing and executing poisoned dependencies without developer oversight.

To characterize this emerging threat, this study presents a Systematic Literature Review (SLR) synthesizing current research across five core dimensions: hallucination prevalence, taxonomies of hallucination, slopsquatting threat mechanics, exploitability assessment, and mitigation strategies. The synthesis indicates that hallucinated dependencies constitute a systemic and persistent flaw, with both commercial and open-source models exhibiting non-trivial hallucination rates. Furthermore, traditional supply-chain defenses and integrity frameworks, such as SLSA and the OpenSSF Scorecard, are not well equipped to detect this class of novel, AI-generated naming deception. Addressing this vulnerability requires a multi-layered defense architecture that combines generation-time grounding (e.g., Retrieval-Augmented Generation), post-generation developer verification within the IDE, and stricter namespace-reservation policies at the registry level. The review concludes by identifying critical gaps in empirical exploitability benchmarking and underscores the need to transition toward a zero-trust model for AI-assisted software development.

Keywords: AI code hallucination, slopsquatting, software supply chain security, large language models, autonomous coding agents, dependency confusion

Contents

Declaration	ii
Abstract	iii
Contents	iv
List of figures	viii
List of Tables	ix
1. Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Objectives and Questions	2
1.4 Document Structure.....	3
2. Literature Review	4
2.1 Systematic Literature Review Methodology.....	4
2.1.1 Search Strategy	4
2.1.2 Boolean Search Strings	6
2.1.3 Inclusion and Exclusion Criteria.....	7
2.1.4 Study Selection Process (PRISMA).....	8
2.1.5 Data Extraction Process	10
2.1.6 Quality Assessment.....	10
2.1.7 Thematic Analysis Framework	11
2.2 AI-Assisted Software Development Landscape.....	12
2.2.1 Evolution of AI Code Generation.....	12
2.2.2 Traditional Programming vs AI-Assisted Programming.....	13
2.2.3 Agentic Coding vs Traditional AI Coding Assistants	14
2.3 Large Language Models for Code Generation.....	16
2.3.1 Commercial LLMs vs Open-source LLMs	16
2.3.2 Hallucination in Code Generation	17
2.3.3 Package Hallucination	19
2.4 Software Supply Chain Security	21
2.4.1 Evolution of Supply Chain Attacks	21
2.4.2 Typosquatting vs Dependency Confusion vs Slopsquatting	22
2.4.3 Package Registry Architectures.....	24

2.4.4 Software Supply Chain Risks in AI-generated Code	25
2.5 Detection Approaches	28
2.5.1 Static Detection Methods.....	28
2.5.2 Runtime Detection Methods.....	29
2.5.3 Static vs Runtime Detection	30
2.6 Hallucination Mitigation Strategies	33
2.6.1 Retrieval-Augmented Generation (RAG).....	33
2.6.2 Knowledge Graph Grounding	35
2.6.3 API Documentation Grounding.....	36
2.6.4 Comparative Analysis: RAG vs Knowledge Graph vs API Documentation	37
2.7 Evaluation of Hallucination Mitigation.....	42
2.7.1 Comparative Evaluation of Hallucination Mitigation Studies	42
2.8 Research Gaps	46
2.8.1 Synthesis of Research Gaps Across the Literature	46
3. Methodology	48
3.1 Introduction and Methodological Philosophy	48
3.1.1 Research Philosophy and Approach	48
3.1.2 Research Design Overview	49
3.3 Study Design and Research Framework	50
3.3.1 Overview of the Multi-Phase Research Design.....	50
3.3.2 Conceptual Framework.....	51
3.3.2.1 Theoretical Foundations.....	53
3.3.2.2 Three-Domain Intersection Model.....	53
3.3.2.3 Three-Stage Threat Model	53
3.3.3 Operationalization of Variables.....	54
3.3.4 Research Hypotheses	57
3.4 Data Collection.....	58
3.4.1 Programming Environments and Ecosystems	58
3.4.2 AI Tools and Models	59
3.4.3 Code Generation Protocol	60
3.4.3.1 Prompt Design and Categorization	60
3.4.3.2 Functional Categories	60

3.4.3.3 Structured Prompt Templates	61
3.4.3.4 Prompt Execution and Response Capture.....	61
3.4.3.5 Metadata Recording	61
3.4.4 Sample Size Determination	62
3.4.5 Data Storage and Organization.....	62
3.5 Dependency Extraction and Hallucination Detection	62
3.5.1 Dependency Extraction Protocol	63
3.5.1.1 Spring Boot Dependency Extraction	63
3.5.1.2 Node.js Dependency Extraction.....	64
3.5.2 Validation Protocol.....	64
3.5.2.1 Maven Central Registry Lookup.....	65
3.5.2.2 npm Registry Lookup	65
3.5.2.3 Manual Verification for Ambiguous Cases	65
3.5.2.4 Documentation and Repository Presence Verification	65
3.5.3 Classification Framework.....	66
3.5.4 Dependency Classification Workflow	66
3.6 Risk Assessment and Exploitability Analysis.....	68
3.6.1 Risk Assessment Dimensions.....	68
3.6.1.1 Registry Exposure	68
3.6.1.2 Naming Similarity Assessment.....	69
3.6.1.3 Functional Criticality	69
3.6.1.4 Cross-Tool Consistency	70
3.6.2 Severity Scoring Framework	70
3.6.3 Exploitability Assessment Methodology	71
3.7 Conceptual Framework and Risk Assessment Model.....	72
3.7.1 Model Development	72
3.7.1.1 Integration of Frequency, Exploitability, and Severity.....	72
3.7.1.2 Model Architecture	72
3.7.2 Model Parameters and Weighting	73
3.7.2.1 Parameter Definition.....	73
3.7.2.2 Weighting Scheme Justification	73
3.7.3 Model Applicability Across Environments	74
3.8 Mitigation Strategy Formulation.....	75

3.8.1 Stakeholder-Specific Recommendations	75
3.8.1.1 Developer-Level Strategies.....	75
IDE Integration Recommendations	76
3.8.1.2 Organizational-Level Strategies.....	76
3.8.1.3 Tool Vendor Recommendations	77
3.8.1.4 Registry Maintainer Recommendations.....	77
3.8.2 Feasibility Assessment	78
3.9 Model Validation.....	79
3.9.1 Internal Validation.....	79
3.9.1.1 Dataset Split Strategy.....	79
3.9.1.2 Predictive Accuracy Assessment	80
3.9.1.3 Inter-Rater Reliability Testing	80
3.9.2 External Validation.....	81
3.9.2.1 Cross-Temporal Validation.....	81
3.9.2.1.1 Temporal Holdout Sets	81
3.9.2.2 Cross-Tool Validation.....	81
3.9.2.3 Expert Review.....	82
3.9.3 Sensitivity Analysis	82
3.10 Data Structure and Documentation	83
3.10.1 Dataset Structure.....	83
3.10.2 Documentation Standards.....	85
3.11 Ethical Considerations.....	86
3.11.1 Live Registry Manipulation Avoidance	86
3.11.2 Responsible Disclosure Considerations.....	86
3.11.3 Research Integrity and Transparency	86
3.11.4 Data Privacy and Security.....	86
3.12 Methodological Limitations	87
3.12.1 Scope Constraints	87
3.12.2 Temporal Snapshot Limitation	87
3.12.3 Sample Size Considerations	88
4. References.....	89

List of figures

Figure 2-1: Prisma Framework	9
Figure 2-2 : Operational dataflow of AI-driven package hallucination propagation and slopsquatting execution within autonomous agent lifecycles.	27
Figure 2-3 : details a multi-stage defensive architecture	33
Figure 2-4 : Operational layout of a multi-layered hybrid mitigation pipeline combining unstructured text vectors, structured knowledge graph oracles, and post-execution runtime validation.....	39
Figure 2-5 : Automated evaluation framework mapping parallel static checking, database grounding lookup, and sandboxed execution telemetry validation.....	45
Figure 3-0-1 : Conceptual Framework Diagram.....	51
Figure 3-2 : Dependency Classification Workflow Pipeline	67

List of Tables

Table 2-1: Evolution of Research Terminology Across Domains	5
Table 2-2 : Comparative Evaluation of Hallucination Mitigation Strategies	40
Table 3-1 : Operationalization of Research Variables	54

1. Introduction

1.1 Background

The integration of Large Language Models (LLMs) into modern software engineering has fundamentally altered the development lifecycle. AI-powered coding assistants and autonomous agents are increasingly used to generate source code, recommend Application Programming Interfaces (APIs), and automate complex programming tasks, with industry estimates suggesting that a substantial and growing share of newly written code is now AI-generated (citation needed). This shift has culminated in paradigms such as *vibe coding*, in which developers articulate high-level intent in natural language while delegating implementation, debugging, and dependency management to autonomous AI agents.

Concurrently, the baseline software supply-chain threat landscape has grown increasingly hostile. Modern software relies heavily on reusable third-party components distributed through public package registries such as npm, PyPI, and Maven Central, creating a substantial attack surface underscored by major real-world incidents, including the SolarWinds, Log4j, and xz-utils compromises. Historically, malicious actors have exploited this surface through human-driven attack vectors such as typosquatting (publishing malware under slightly misspelled versions of popular package names) and dependency confusion (exploiting namespace-resolution ambiguity between private and public registries). These attacks aim to deceive developers or build systems into inadvertently downloading and executing malicious code, resulting in severe upstream and downstream compromise across the software supply chain.

1.2 Problem Statement

Despite their capabilities, code-generating LLMs are subject to a critical flaw known as **hallucination** the generation of output that is syntactically valid and fluent but factually incorrect, logically unsound, or unsupported by real-world context. In the context of software development, this manifests specifically as package hallucination, in which a

model confidently recommends or imports non-existent software libraries, modules, or API endpoints.

This generative flaw has introduced a novel and highly exploitable vulnerability into the software supply chain. Adversaries monitor LLM outputs to identify frequently hallucinated package names and pre-emptively register them on public open-source registries with malicious payloads. This attack vector, termed slopsquatting, differs from traditional typosquatting in that it depends entirely on AI fabrication rather than a human developer's typographical error. The risk is substantially amplified by autonomous coding agents with system-level execution privileges, which can act as confused deputies that download and execute maliciously registered phantom dependencies without human verification. As a result, LLMs can inadvertently function as delivery mechanisms for supply-chain poisoning, transforming a routine generation error into a serious security compromise.

1.3 Research Objectives and Questions

Given the intersection of AI fallibility and software supply-chain vulnerability, this literature review systematically investigates the nature, exploitability, and mitigation of package hallucination in LLM-generated code. Specifically, the review aims to identify the underlying mechanisms driving these errors and to consolidate defensive strategies for securing AI-assisted development workflows. The study is guided by the following research questions:

- **RQ1:** What are the root causes and prevalence of dependency hallucination in code-generating LLMs? This question examines how model architecture, training data constraints, and probabilistic inference contribute to the fabrication of APIs and packages across different programming ecosystems.
- **RQ2:** How do slopsquatting attacks manifest, and what distinguishes them from traditional software supply-chain threats? This question examines the transition from benign code hallucination to weaponized supply-chain compromise, with particular attention to the developer "comprehension gap" and the role of agentic autonomy.

- **RQ3:** What multi-layered mitigation strategies exist to detect and prevent AI-induced supply-chain attacks? This question examines current defense mechanisms spanning generation-time grounding, IDE-based validation pipelines, and registry-level policy enforcement.

1.4 Document Structure

The remainder of this paper is organized as follows. Section 2 details the systematic literature review methodology, including the search strategy, data sources, and inclusion criteria. Section 3 presents the thematic synthesis of the reviewed literature across five core dimensions: prevalence and ecosystem dynamics, taxonomies of code hallucination, the slopsquatting threat model, exploitability assessment, and multi-layered mitigation strategies. Section 4 discusses the key findings, considers practical implications, and identifies critical gaps in the current research landscape. Section 5 concludes the paper with a summary of the urgency of this problem and recommendations for securing the future of AI-assisted software development.

2. Literature Review

2.1 Systematic Literature Review Methodology

The objective of this systematic literature review is to establish a transparent, reproducible, and rigorous framework for identifying, evaluating, and synthesizing research relevant to artificial intelligence assisted code generation, software package hallucinations, software supply chain security, and modern vulnerability mitigation approaches. By adhering to formal review protocols, this methodology minimizes selection bias, ensures comprehensive coverage of both academic literature and verified industry threat reports, and provides a stable, empirical foundation for the subsequent thematic synthesis.

2.1.1 Search Strategy

The identification of relevant literature relied on a multi-database search strategy designed to balance historical context with the rapid evolution of large language models. The primary academic indexing databases selected for this review include IEEE Xplore, the ACM Digital Library, Web of Science, SpringerLink, Scopus, and ScienceDirect. These electronic platforms represent the standard repository infrastructure for peer-reviewed computer science and software engineering research. To address the latency associated with formal journal indexing delays and to capture contemporary threat disclosures, this strategy explicitly integrates grey literature from reputable open preprint repositories such as arXiv, alongside high-quality industrial security reports published by established cybersecurity institutions including Trend Micro, Kaspersky, and Sonatype.

A significant challenge in executing this search strategy arises from the fluid and shifting vocabulary across the artificial intelligence and software security domains. While traditional software supply chain literature focuses extensively on established mechanisms such as typosquatting and dependency confusion, emerging studies on generative artificial intelligence introduce novel terminologies including package hallucination, phantom dependencies, and slopsquatting. This division represents a

fundamental research gap, as indexing systems frequently fail to align cross-domain publications that bridge artificial intelligence flaws with cybersecurity exploits. Consequently, the search strategy incorporates both traditional terminology and emerging definitions to guarantee complete evidence coverage up to the current literature horizon. The search period was specifically selected to align with the post-2020 evolution of LLM-based coding assistants, ensuring historical baselines are captured alongside rapid contemporary architectural shifts.

To ensure an optimal balance between retrieval sensitivity and precision, structured search configurations were designed to encompass the structural breadth of the domain.

To complement database selection, the evolution of search terminology reflects the historical transition from manual software supply chain risks to automated generative flaws. Table 2.1 maps this structural shift, highlighting traditional, transitional, and emerging domain terms.

Table 2-1: Evolution of Research Terminology Across Domains

Focus	Core Vocabulary	Target Mechanism	Foundational Source Context
Traditional Security	Typosquatting, Dependency Confusion	Registry Deception via Human Error	Ecosystem Vulnerabilities
Transitional AI	Code Hallucination, Factual Inconsistency	Model Semantic Fabrication	LLM Generation Flaws
Emerging Synthesis	Slopsquatting, Package Hallucination	Automated Exploitation of AI Vulnerabilities	Generative Supply Chain Attacks

The distribution of candidate literature spans multiple publication types, with initial data indicating that while peer-reviewed conferences provide established empirical validation framework benchmarks, open preprint repositories host the highest volume of timely documentation regarding active exploit vectors. To achieve operational

replicability, these conceptual domains must be operationalized into query code, as detailed in the following subsection.

2.1.2 Boolean Search Strings

To avoid retrieving thousands of irrelevant articles concerning general natural language processing or unrelated software bugs, a carefully targeted search strategy was constructed. Using basic terms like "AI code vulnerabilities" would yield an overwhelming number of results focused on traditional static analysis or prompt injection, which do not reflect the specific generative failures and dependency risks associated with AI package fabrication. Therefore, three distinct, progressive Boolean search strings were developed. These queries narrow the literature down systematically, moving from the broad intersection of artificial intelligence and software security toward the highly specific supply-chain exploitation mechanisms central to this review.

The three progressive search strings utilized were:

- **String 1:** Baseline Context ("Large Language Models" OR "LLM" OR "Generative AI" OR "AI coding assistants") AND ("software engineering" OR "code generation" OR "programming") AND ("security" OR "vulnerability" OR "threat") Justification: This foundational string aimed to capture the broad landscape of security implications within AI-assisted software development. Including general security terminology ensured the retrieval of essential background literature regarding how large language models alter the traditional developer threat model, regardless of the specific vulnerability type.
- **String 2:** The Generative Flaw ("Large Language Models" OR "LLM" OR "AI assistants") AND ("code generation" OR "API integration" OR "dependency management") AND ("hallucinat*" OR "fake API*" OR "fabricated library" OR "phantom symbol*") Justification: This string built upon the first by narrowing the focus from general security flaws to the specific phenomenon of model hallucinations in code. By filtering for "fake APIs" and "phantom symbols," this query successfully excluded irrelevant medical or linguistic hallucination papers, strictly isolating research that examines how models invent non-existent software components.
- **String 3:** The Exploitation Vector ("AI-generated code" OR "LLM" OR "autonomous agents") AND ("package hallucination" OR "slopsquatting" OR "malicious package*") AND ("supply chain" OR "dependency confusion" OR "package registry") Justification: This final, highly targeted string addressed the ultimate security impact of code hallucinations. It captured cutting-edge threat intelligence regarding how adversaries weaponize these generative

flaws. By focusing on terms like "slopsquatting" and "supply chain," this query isolated the literature detailing how fake dependencies are actively registered in open-source ecosystems (e.g., npm, PyPI) to execute stealthy supply-chain compromises.

Applying these three progressively narrower search strings allowed for a rigorous filtering process. It effectively eliminated theoretical discussions on general AI safety while capturing the precise technological evolution from a simple generative coding error (hallucination) to a systemic cybersecurity threat (supply chain poisoning).

2.1.3 Inclusion and Exclusion Criteria

To ensure a non-biased, reproducible selection process, clear study selection criteria were defined to filter the raw query output. The application of these criteria guarantees the retention of empirical studies directly addressing the crossover between artificial intelligence generation flaws and software supply chain vulnerabilities.

Inclusion Criteria:

- The study must be written entirely in the English language.
- The paper must present empirical data, documented case studies, or architectural innovations explicitly addressing code generation, package hallucinations, or semantic vulnerabilities in large language models.
- The publication must analyze security-relevant consequences, attack vectors, or technical mitigation mechanisms within software supply chains.
- The study must evaluate defined software package registries or developer ecosystem workflows.

Exclusion Criteria:

- Purely conceptual studies, non-evaluated tool proposals, editorials, or anecdotal blog posts lacking formal experimental validation are excluded.
- Conference publications that have been subsequently expanded into comprehensive journal versions are omitted to avoid duplicate counting.
- Studies focusing exclusively on general natural language translation, productivity gains, or developer velocity enhancements of AI coding helpers, without assessing security degradation, package fabrication, or correctness failures, are excluded.

- Duplicate records across indexing platforms.

A persistent operational challenge in this phase involves ambiguous study classifications, particularly when dealing with preprints that undergo rapid iterative updates on arXiv. Furthermore, publication bias represents a fundamental methodological limitation, as researchers frequently emphasize successful mitigation results while omitting complete reporting on edge-case experimental failures or minor model deviations. Boundary decisions regarding borderline study relevance—such as papers evaluating general software vulnerabilities without an explicit artificial intelligence context—require a structured multi-reviewer arbitration process to maintain screening consistency. The systematic execution of these criteria under standard screening phases is detailed next.

2.1.4 Study Selection Process (PRISMA)

The screening and final selection of candidate literature adhered strictly to the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines. This iterative process tracks the exact attrition rate of retrieved papers across four distinct phases: identification, screening, eligibility assessment, and final inclusion.

During the initial identification phase, automated query execution across the selected academic databases and preprint repositories yields the raw candidate corpus.

Duplicate records are systematically compiled and removed using reference management software. Next, the title and abstract screening phase filters out completely off-topic papers, such as those addressing hallucinations in medical imaging or time-series economic forecasting models.

The remaining studies undergo a thorough full-text eligibility review. The principal cause for attrition at the full-text stage involves studies that evaluate general natural language flaws rather than executable software code semantics, or papers addressing human typosquatting without considering automated generative risks. To ensure screening consistency and high inter-rater reliability, multiple reviewers piloted the criteria, utilizing tie-breaking procedures to resolve subjective eligibility decisions.

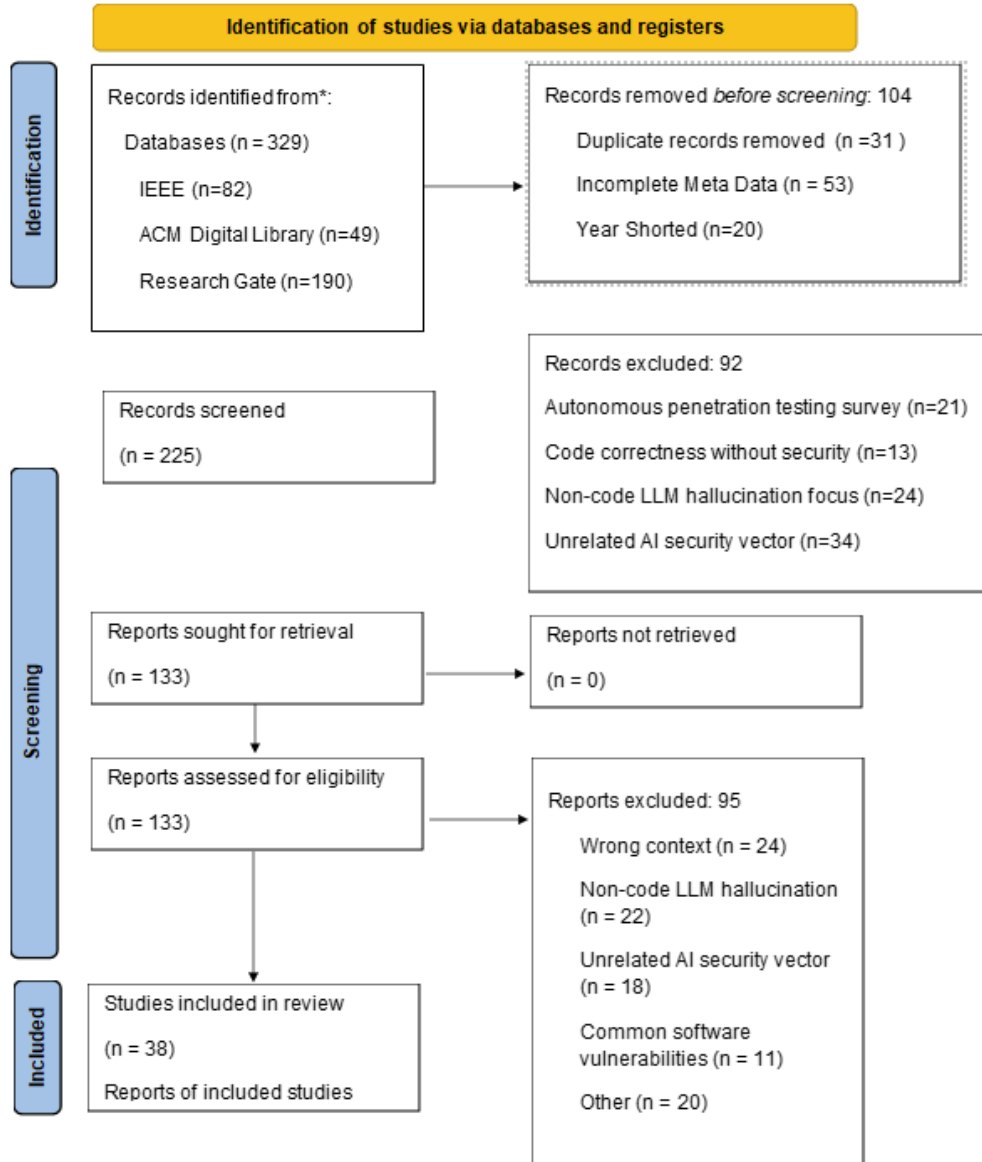


Figure 2-1: Prisma Framework

The conversion rates from retrieved to included studies vary significantly across platforms; high-precision indexes yield lower total counts but higher final inclusion percentages, whereas preprint servers present high initial volume alongside steep attrition rates during full-text review. Following the finalization of the included paper corpus, the standardized extraction of empirical data was performed as outlined in the next subsection.

2.1.5 Data Extraction Process

A standardized data extraction framework was applied to the finalized literature base to isolate publication details, experimental variables, modeling techniques, and performance metrics. The extraction matrix focuses heavily on isolating the architectural characteristics of the evaluated large language models alongside the security profile of the targeted language ecosystems.

An analysis of the empirical patterns across rows reveals a clear performance divergence between proprietary, commercial model architectures and open-source implementations, with smaller open-source parameters consistently exhibiting higher baseline hallucination rates across diverse tasks. Furthermore, the synthesis demonstrates that evaluation metrics are highly sensitive to model temperature settings and target API invocation frequencies, as seen in the elevated error rates for low-frequency APIs and maximum generation temperatures. A severe methodological limitation across the existing literature is heterogeneous reporting; many studies omit uniform dataset descriptions, leave mitigation variables unspecified, or rely on distinct definitions of evaluation metrics, which complicates cross-study comparison. The extracted data requires systematic evaluation to judge its empirical robustness, which is addressed through quality assessment in the next subsection.

2.1.6 Quality Assessment

The methodological quality and validity of the selected primary studies were systematically evaluated using a multi-dimensional assessment framework. The criteria evaluate each study on its computational reproducibility, dataset transparency, depth of evaluation, and formal acknowledgment of threats to validity.

A critical comparison reveals distinct quality variations between traditional peer-reviewed literature and rapid preprint releases. Peer-reviewed papers generally demonstrate higher experimental rigor, larger sample distributions, and documented disclosure processes. Conversely, early preprints frequently present strong performance claims or novel tool architectures backed by limited evaluation data, narrow, non-public validation repositories, or single-task configurations that fail to guarantee long-term error correction stability. Furthermore, a methodological gap exists between pure artificial intelligence evaluation studies which prioritize static

metrics like functional correctness and cybersecurity evaluations, which assess runtime exploitability and supply chain implications.

A major open challenge identified during this quality assessment is the difficulty of verifying reproducibility. Many studies fail to release full prompt sets, temperature configurations, or raw model completion outputs, complicating the exact validation of hallucination behavior over time. This is compounded by a lack of standardized vulnerability data-interchange formats across provenance frameworks and the geographical or ecosystem clustering of evaluation tests, which often focus exclusively on Python environments while lacking dynamic or ecosystem-wide language validation. The qualitative groupings derived from these studies are organized structurally using the thematic framework described in the final subsection.

2.1.7 Thematic Analysis Framework

The synthesis of the qualified literature is structured around a multi-axial thematic analysis framework designed to align directly with the core research questions of this thesis. Studies were grouped conceptually into four distinct research themes to provide a clear, logical progression through the literature:

- **Theme 1: Prevalence and Ecosystem-Specific Dynamics (Corresponds to Sections 2.2 and 2.3):** Quantifying hallucination and vulnerability rates across commercial and open-source models, while contrasting namespace validation behaviors between flat registries (npm, PyPI) and domain-verified infrastructures (Maven Central).
- **Theme 2: Taxonomies and Generative Mechanisms (Corresponds to Section 2.4):** Categorizing code hallucinations into knowledge-conflicting, context-deviating, or intent-conflicting errors, and mapping the evolutionary shift from human motor errors (typosquatting) and configuration flaws (dependency confusion) to deterministic model failures (slopsquatting).
- **Theme 3: Autonomous Exploitation and Threat Modeling (Corresponds to Sections 2.2.3 and 2.4.4):** Assessing how reasoning-enhanced coding agents, autonomous tool environments (Cursor Agent, Claude Code, OpenHands, Devin), and agentic vibe coding workflows modify the velocity, execution privileges, and shell access risks of supply chain compromise.

- **Theme 4: Multilayered Mitigation Strategies and Evaluation**
(Corresponds to Sections 2.5, 2.6, and 2.7): Evaluating the technical trade-offs, processing overheads, false-positive metrics, and structural failure points of external knowledge grounding (RAG, Knowledge Graphs, API Documentation) versus static and runtime detection engines.

A critical tension within this framework exists between attack-focused research, which empirically documents high vulnerability rates and registry bypasses, and mitigation-focused studies, which often report optimization metrics under restricted, offline experimental settings. Current literature heavily privileges flat registries like npm and PyPI due to their high open-source sample availability, leaving a research gap in cross-ecosystem evidence for enterprise environments utilizing scoped or verified domains. Concluding this section, the defined thematic framework provides the necessary structure for synthesizing the substantive literature reviewed in the remainder of Chapter 2.

2.2 AI-Assisted Software Development Landscape

2.2.1 Evolution of AI Code Generation

The paradigm of automated software development has shifted from token-level autocomplete mechanisms toward repository-level generation frameworks. This trajectory highlights a fundamental tension regarding how source code is modeled, with contemporary paradigms emphasizing that code should not be treated merely as natural language due to the distinct structural and syntactic complexities inherent to multi-file repositories [Le-Anh et al., 2026]. Systematic literature reviews map this evolution by tracking how expanding model capabilities scale alongside the severity of generative flaws, characterizing a broad taxonomy of code hallucinations that manifest across modern large language models [Daoud, 2026]. While early research focused on token-level predictions, contemporary evaluations increasingly address multi-line and structural code generation anomalies [Liu et al., 2026].

A primary point of divergence in the literature concerns the operational trade-offs between commercial, proprietary large language models and specialized open-source coding variants. Proprietary architectures leverage massive parameter scales to support broad conversational capabilities, yet systematic analyses reveal that these models remain highly susceptible to code-specific hallucinations [Gao et al., 2025]. Although general-purpose models are widely integrated into professional development environments, their underlying behavioral patterns display significant variations when tasked with specialized language syntax, such as inventing non-existent packages or misinterpreting structural semantics [Zheng et al., 2026].

Adoption statistics show that up to 97% of developers now incorporate generative AI into their workflows, and approximately 30% of codebase compositions contain AI-generated contributions [Al-Zofi, 2025]. However, a distinct gap within this body of literature is the lack of longitudinal datasets tracking user satisfaction scores, productivity deltas, or enterprise deployment friction between specific commercial assistants such as GitHub Copilot, ChatGPT, Claude, Gemini, Cursor, and Windsurf. The reviewed literature does not address the cost implications, long-term scalability, or proprietary intellectual property protections associated with deploying these models across enterprise-scale repositories, leaving these as unresolved challenges in current software engineering literature. Despite widespread industry integration, empirical evidence remains mixed regarding the overall benefits and risks of AI-assisted programming compared with traditional software development practices.

2.2.2 Traditional Programming vs AI-Assisted Programming

Integrating generative models into the software development lifecycle alters the established mechanics of code construction, debugging, and verification. In traditional programming workflows, verification relies entirely on deterministic test suites, manual code reviews, and human oversight. Conversely, AI-assisted programming introduces non-deterministic generative code that requires continuous trust calibration and validation [Liu et al., 2026]. This structural shift is explored through automated frameworks designed to identify discrepancies between generated code and intended functionality, such as using execution-based verification to expose code hallucinations that bypass static syntax checks [Tian et al., 2025]. The necessity for explainable and

reliable debugging paradigms is amplified in production environments, where the opacity of deep-learning models complicates error isolation and structural maintenance [Dubey and Madiseti, 2026].

The transition from manual coding to collaborative, AI-assisted programming also reframes the risks associated with API integration and interface alignment. Models frequently generate code that contains severe API misuse or relies on fabricated interface specifications [Zhuo et al., 2025]. To mitigate these systemic errors, architectural interventions focus on grounding model context with real-time API documentation, which structurally reduces the frequency of interface hallucinations compared to ungrounded conversational generation [Jain et al., 2025]. However, systematic reviews of security-related code hallucinations indicate that even when syntax appears correct, the generated code frequently introduces subtle security vulnerabilities that escape manual human inspections [Daoud, 2026].

A major methodological limitation across these comparative studies is the failure to evaluate how performance varies across different developer experience levels. The reviewed literature does not provide empirical comparisons distinguishing how novice developers handle hallucinated APIs versus how experienced engineers recalibrate their trust when encountering flawed AI recommendations. Additionally, while grounding methods like API documentation insertion or knowledge graph reasoning are shown to minimize errors [Li et al., 2026], the texts do not quantify the specific latency overhead, computational costs, or execution bottlenecks introduced by these real-time context-expansion techniques in industrial settings, which represents a critical research gap. Current AI assistants increasingly perform tasks beyond code generation, motivating the emergence of autonomous agentic coding systems.

2.2.3 Agentic Coding vs Traditional AI Coding Assistants

The contemporary software engineering landscape is defined by the migration from traditional, reactive AI coding assistants to autonomous agentic coding systems. Traditional assistants function within a strict human-in-the-loop constraint, executing localized autocomplete prompts or answering standalone conversational queries without direct environment interaction. In contrast, modern agentic systems possess tool-use capabilities, multi-step reasoning, and varying degrees of operational

autonomy, enabling them to execute shell commands, modify local file systems, and interact directly with external package registries [Gandhi, 2025]. This transition dramatically amplifies systemic security risks within autonomous development workflows and vibe-coding environments, as the removal of mandatory human verification points exposes the underlying architecture to unauthorized code execution and automated exploit vectors [Yadav et al., 2026].

To counter the supply-chain risks introduced by autonomous development, recent research has focused on quantifying this threat, revealing that package hallucinations are a persistent and systemic phenomenon across state-of-the-art code-generating LLMs [Spracklen et al., 2024]. Furthermore, the broader classification of AI-assisted software development as a novel supply-chain attack surface highlights the fact that traditional security taxonomies fail to capture the risks associated with non-deterministic agentic actions [Yadav et al., 2026]. A critical overview of attacks on open-source supply chains underscores the necessity of mapping these novel vectors against historical attack methodologies like typosquatting and dependency confusion [Ohm et al., 2020].

A profound research gap in this domain involves the lack of deployment data regarding the computational and operational overhead of these advanced defense mechanisms. For instance, while program regeneration is proposed to neutralize supply-chain attacks, the literature provides no analysis regarding its execution latency, resource scalability, or performance degradation when applied to enterprise-scale production codebases. Moreover, there is an absence of comparative benchmarks evaluating leading agentic systems such as Cursor Agent, Claude Code, OpenHands, Devin, and GitHub Copilot Agent in terms of their safety boundaries, autonomous package installation success rates, or human intervention frequencies under adversarial conditions. The increasing autonomy significantly expands both the capabilities and risks of AI-assisted software development, naturally leading to a detailed examination of AI-generated code reliability, package hallucinations, and software supply chain security threats in the following sections.

The empirical findings establish a clear pattern showing that hallucination rates scale substantially higher under high temperature settings or when targeting low-frequency API segments. The data highlights Twist et al. (2026) as a prominent outlier, where

specific temporal prompts drove dependency fabrication rates up to 85%. This variance underscores a severe fragmentation in evaluation protocols, where mitigation strategies such as RAG and self-refinement show localized accuracy improvements but suffer from significant computational overhead and scale poorly against real-world enterprise architectures.

2.3 Large Language Models for Code Generation

2.3.1 Commercial LLMs vs Open-source LLMs

The comparative landscape of automated code generation is characterized by a structural division between commercial, proprietary large language models and open-source, self-hosted architectures. Proprietary foundational models such as GPT-4o, Claude, and Gemini demonstrate high functional correctness on standard execution benchmarks, yet their closed nature introduces severe challenges for scientific reproducibility and model transparency [Liu et al., 2026]. Conversely, open-source and self-hosted models, including the DeepSeek-Coder series, Qwen-Coder architectures, Llama foundations, and CodeLlama specialized variants, offer complete transparency regarding weights and architectural parameters, but frequently exhibit greater performance variability depending on the target ecosystem and programming language syntax [Zheng et al., 2026]. Empirical evaluations on coding benchmarks such as HumanEval, MBPP, SWE-Bench, and LiveCodeBench reveal significant variations in coding accuracy; while certain studies demonstrate the clear superiority of high-parameter commercial models like GPT-4o, conflicting evidence shows that open-source specialized models such as Qwen2.5-Coder and DeepSeek-Coder-V2 can achieve comparable or even superior functional correctness under optimized prompting strategies [Twist et al., 2026; Zheng et al., 2026]. This tension highlights a critical methodological limitation within current literature: observed performance differentials are frequently artifacts of benchmark selection, evaluation datasets, and specific prompting paradigms rather than intrinsic model capabilities [Lian et al., 2024].

Furthermore, while commercial providers emphasize the general-purpose capability of their architectures, code-specialized models are explicitly optimized for syntactical regularity and structural dependencies [Jain et al., 2025]. However, the reviewed literature presents a critical gap regarding cost-performance trade-offs and enterprise

deployment constraints. While proprietary models incur continuous API operational costs, self-hosted open-source deployments require massive local computational infrastructure, a trade-off that remains unquantified and unaddressed within existing empirical studies. Security vulnerability generation studies further polarize the commercial versus open-source dichotomy. Closed-source models often incorporate advanced safety alignment and reinforcement learning from human feedback to suppress the generation of insecure code patterns, whereas open-source architectures exhibit varying degrees of vulnerability injection, often recreating historical software defects found in their unvetted training corpora [Daoud, 2026]. Package recommendation reliability similarly diverges; a systematic analysis across 2.23 million code samples reveals that commercial models maintain a lower dependency fabrication rate of approximately 5.2%, whereas open-source models demonstrate a substantially higher rate of 21.7% [Al-Zofi, 2025]. Despite these clear empirical discrepancies, a cross-cutting limitation is the lack of standardized cross-ecosystem evaluations, as current comparative datasets heavily prioritize mainstream platforms like Python and JavaScript, leaving compiled or enterprise languages under-examined. Despite differences in overall coding performance, hallucination remains a common limitation across both commercial and open-source LLMs, motivating a focused synthesis of hallucination research.

2.3.2 Hallucination in Code Generation

Code hallucinations are widespread across generative architectures, presenting deep vulnerabilities to software correctness, maintainability, and security [Gao et al., 2025]. The literature reveals a fundamental tension regarding the primary root causes of code fabrication. One school of thought attributes code hallucinations to context window limitations and model reasoning constraints during long-sequence generation, suggesting that models default to probabilistic fabrication when local context becomes exhausted [Tian et al., 2025]. An opposing perspective argues that hallucinations stem primarily from training data deficiencies, data noise, and exposure bias, where the model mimics unvetted or contradictory patterns embedded within its pre-training corpora [Gao et al., 2025]. This cause-centric debate extends to the relationship between model scale and error rates. While a subset of the literature suggests that scaling model parameters inherently minimizes generative errors by expanding internal factual

representations, conflicting empirical evidence demonstrates that larger models frequently display higher overconfidence and can hallucinate complex, nonexistent API entities at rates comparable to or exceeding smaller models when confronted with semantic ambiguity or low-frequency programming tasks [Tripathi et al., 2025].

To mitigate these generative flaws, current architectural interventions contrast standalone conversational LLMs against retrieval-enhanced approaches, specifically comparing Retrieval-Augmented Generation, Knowledge Graph reasoning, and verified API documentation grounding [Daoud, 2026; Li et al., 2026]. Standalone conversational models suffer from strict temporal cut-offs and are incapable of aligning with evolving software specifications [Twist et al., 2026]. While grounding models with real-time API documentation reduces invocation errors, documented failure modes reveal that sub-optimal retrievers frequently inject irrelevant context, which actively degrades model performance on high-frequency, well-known APIs, introducing a 39.02% absolute drop in correctness [Jain et al., 2025]. Furthermore, the reviewed literature does not address the computational overhead, system latency, or enterprise scalability constraints associated with running complex real-time knowledge graph or retrieval-augmented pipelines within continuous integration and continuous deployment environments, marking a distinct gap in the technical literature.

The evaluation methodologies employed to detect and measure code hallucinations represent another core tension. Historical frameworks relied on match-based evaluation metrics such as Exact Match or CodeBLEU, which quantify text-level similarity against a single reference answer [Lian et al., 2024]. However, empirical analyses show that match-based evaluation methods fail to accommodate valid implementation variability, causing exceptionally high false-negative rates such as misclassifying up to 43.66% of correct GPT-4 outputs on datasets like CoNaLa [Lian et al., 2024]. To overcome this blind spot, contemporary methodologies utilize execution-based verification, which evaluates functional correctness by executing generated code against dynamic test suites [Tian et al., 2025; Yang et al., 2026]. Dynamic execution algorithms like Code-Halu expose hidden logic, mapping, and resource hallucinations that pass basic static syntax checks [Tian et al., 2025]. Nonetheless, a persistent open challenge is the lack of standardized hallucination definitions and the extreme difficulty in distinguishing pure reasoning failures from factual knowledge hallucinations. Moreover, current evaluation benchmarks remain highly fragmented and rely on synthetic snippets rather than real-

world, repository-level software engineering datasets, with an absolute absence of longitudinal studies tracking the decay of developer trust over prolonged exposure to hallucinated code suggestions. Conclude that package hallucination represents one of the most security-critical manifestations of code hallucination and requires dedicated investigation.

2.3.3 Package Hallucination

Package hallucination represents a security-critical manifestation of code fabrications, transforming probabilistic generative anomalies into tangible vectors for software supply chain exploitation [Al-Zofi, 2025]. This risk surface is structurally defined by the convergence of non-deterministic LLM package recommendations and public registry architectures [Spracklen et al., 2025]. Adversaries actively exploit this intersection through slopsquatting, which refers to the malicious pre-registration of AI-hallucinated dependency names on public package managers [Al-Zofi, 2025]. The literature carefully delineates slopsquatting from traditional supply chain threats such as typosquatting and dependency confusion. While typosquatting relies on manual human typing errors and dependency confusion manipulates internal namespace resolution hierarchies to supersede private packages with public entries, slopsquatting exploits the linguistic fluency and factual inaccuracy of generative code recommendations [Al-Zofi, 2025]. Because developers often maintain a high baseline trust in AI assistants, hallucinated imports are easily overlooked, allowing malicious code to propagate silently into downstream production environments [Al-Zofi, 2025]. Empirical data reveals profound variations in package hallucination rates across specific ecosystems, highlighting distinct architectural risks between flat namespaces and domain-verified registries. Ecosystems operating under flat namespaces, such as Python's PyPI and JavaScript's npm registries, demonstrate an exceptionally high vulnerability surface [Al-Zofi, 2025]. Large-scale evaluations show that package hallucination frequencies across popular LLMs reach approximately 15.8% for Python and 21.3% for JavaScript [Spracklen et al., 2025]. Conversely, domain-verified registries such as Java's Maven Central exhibit near-zero package hallucination rates, an ecosystem-specific resilience driven directly by strict group identifier validation and rigorous namespace registration requirements [Al-Zofi, 2025; Spracklen et al., 2025]. However, significant contradictions exist regarding the exact frequency and drivers of

dependency fabrications across studies. For instance, while massive multi-model evaluations report an average package hallucination rate of 19.7% across 576,000 samples [Spracklen et al., 2025], micro-benchmarks targeting temporal boundaries report extreme dependency fabrication rates up to 85% when prompts involve specific time-based constraints or low-frequency API segments [Jain et al., 2025; Twist et al., 2026]. These discrepancies are driven by variations in prompting strategies, dataset choices, and model temperature settings, with package fabrication rates scaling sharply higher alongside sampling randomness [Spracklen et al., 2025].

A primary open challenge identified in this domain is the complete lack of standardized package hallucination benchmarks that span multiple languages and ecosystems uniformly, as the existing literature remains overwhelmingly restricted to Python and JavaScript registries [Yang et al., 2026]. Furthermore, there is a severe deficit of empirical evidence regarding real-world developer behavior when encountering hallucinated dependencies; the literature fails to analyze how frequently a developer installs a phantom package without checking its validity. Methodologically, current security frameworks lack automated mechanisms capable of detecting fabricated packages prior to installation, and the long-term systemic security implications of persistent slopsquatting across global open-source code repositories remain completely unmodeled. Package hallucinations directly expose developers to software supply chain attacks, providing the foundation for the next section on software package ecosystems and dependency-based attack techniques.

The empirical findings establish a clear pattern showing that hallucination rates scale substantially higher under high temperature settings or when targeting low-frequency API segments. The data highlights Twist et al. (2026) as a prominent outlier, where specific temporal prompts drove dependency fabrication rates up to 85%. This variance underscores a severe fragmentation in evaluation protocols, where mitigation strategies such as RAG and self-refinement show localized accuracy improvements but suffer from significant computational overhead and scale poorly against real-world enterprise architectures.

2.4 Software Supply Chain Security

2.4.1 Evolution of Supply Chain Attacks

The structural configuration of software development has shifted threat modeling vectors from downstream application security boundaries to upper-stream software supply chain intersections [Al-Zofi, 2025]. Historically, open-source software security assumed that vulnerabilities were primarily accidental bugs introduced during manual code construction. In contemporary distributed architectures, a typical open-source project incorporates an average of 180 direct and transitive third-party dependencies, effectively expanding the potential entry points for supply chain infiltration [Ohm and Stuke, 2023]. Attacker strategies have consequently transitioned from targeting developer infrastructure or compromising active maintainer credentials to exploiting the automated, non-deterministic mechanisms embedded within modern engineering environments [Yadav et al., 2026].

This evolution is accelerated by the widespread adoption of repository-level code generation models that actively suggest multi-file configurations and package integrations [Yadav et al., 2026]. Traditional naming deception attacks required manual intervention by a human threat actor to engineer compliance flaws or intercept install commands. In contrast, AI-driven development introduces complex context surfaces that are vulnerable to automated context poisoning, cross-file context manipulation, and indirect prompt injection attacks [Yadav et al., 2026]. These techniques target the data streams processed by code-generating assistants, forcing the automated output to incorporate insecure components or select compromised library names without explicit human prompt interaction [Yadav et al., 2026].

A central tension in the reviewed literature involves how emerging AI-induced dependency vectors are classified. One segment of researchers argues that slopsquatting represents a fundamentally distinct category of supply chain attack, as it is triggered by non-deterministic model failures and probabilistic fabrications native to deep-learning architectures [Al-Zofi, 2025]. Conversely, an opposing viewpoint characterizes slopsquatting merely as an automated, high-velocity extension of historical package

squatting and confusion techniques, matching the same lifecycle pathways found in traditional registry poisoning [Gandhi, 2025].

This conceptual division reveals an explicit research gap: current literature lacks longitudinal empirical studies tracking the real-world evolution and active occurrence of slopsquatting campaigns within public package indexes over extended temporal scales. Furthermore, existing frameworks fail to present predictive methodologies for how future iterations of reasoning-enhanced or multi-step autonomous agents will modify the exploitability of upstream code suggestions, leaving the long-term threat profile of AI-assisted supply chains unmapped. Although these attacks target software dependencies, their origins, exploitation methods, and security implications differ substantially, requiring a detailed comparative analysis.

2.4.2 Typosquatting vs Dependency Confusion vs Slopsquatting

securing contemporary development pipelines requires a strict architectural comparison between typosquatting, dependency confusion, and slopsquatting, as each technique exploits a separate operational vulnerability within the dependency resolution lifecycle [Al-Zofi, 2025]. Typosquatting relies completely on human motor and typography errors, capitalizing on instances where a developer manually inputs a misspelled package name during command-line installation [Al-Zofi, 2025]. Dependency confusion targets corporate configuration flaws and build system route hierarchies, manipulating internal namespace resolution to trick package managers into pulling an external, maliciously registered public dependency over a private, internal package sharing an identical identifier [Al-Zofi, 2025]. Conversely, slopsquatting completely bypasses both human typographical mistakes and network configuration parameters, directly leveraging the linguistic fluency and factual inaccuracy of generative package hallucinations embedded within AI-recommended code snippets [Al-Zofi, 2025].

This divergence redefines the structural threat model by shifting the attack trigger from human execution mistakes to automated model fabrications. The risk is compounded by the emergence of scaffolding hallucinations, wherein an LLM generates the entire context block including nonexistent imports, phantom constants, and invented method

chains required to execute an imaginary library call [Tileria et al., 2026]. Because the resulting source code appears syntactically correct and aligns with the expected logical flow, developers frequently succumb to automation bias and accept these phantom symbols without manual validation [Al-Zofi, 2025].

Empirical evidence regarding detection success rates reveals that traditional metadata filtering and static syntax scanning struggle to identify slopsquatting attempts before a dependency installation is triggered. Automated defense suites that integrate static checks and behavioral alignment layers show significant performance degradation when testing non-deterministic inputs; evaluations indicate that up to 33.5% of automated supply chain poisoning payloads successfully bypass combined static detection layers [Qu et al., 2026]. Furthermore, approximately 2.5% of malicious cases are documented to completely evade both strict structural alignment rules and localized static defense code blocks [Qu et al., 2026].

This performance barrier exposes a major limitation across existing supply chain studies. Current literature is restricted by a total absence of standardized slopsquatting datasets capable of isolating repeatable, model-embedded dependency fabrications from random sampling variance. Additionally, the technical texts do not provide empirical user studies investigating real-world developer behavior when interacting with phantom symbols, creating an unresolved gap regarding how easily engineers can be deceived into downloading unverified code components during rapid development cycles. Attack feasibility depends not only on attacker strategy but also on the design and security mechanisms of package registries.

The empirical findings establish a clear pattern showing that hallucination rates scale substantially higher under high temperature settings or when targeting low-frequency API segments. The data highlights a severe fragmentation in evaluation protocols, where mitigation strategies such as retrieval-augmented grounding show localized accuracy improvements but suffer from significant computational overhead and scale poorly against real-world enterprise architectures. The extremely low recall score of 0.02 reported by Woesle et al. (2025) demonstrates that relying on a model's internal self-verification is completely insufficient for identifying supply chain threats.

2.4.3 Package Registry Architectures

The operational success of a supply chain poisoning attempt is heavily governed by the namespace configuration and publication boundaries of the target package registry [Al-Zofi, 2025]. The literature draws a stark architectural contrast between domain-verified package registries and open, frictionless ecosystems. Java's Maven Central repository implements a mathematically and hierarchically structured domain-verification scheme that strictly bounds publication rights to namespaces verifiably controlled by the developer, effectively neutralizing naming deception and squatting attempts [Al-Zofi, 2025]. In contrast, Python's PyPI and JavaScript's npm operate on flat, first-come-first-served naming structures that completely omit mandatory pre-publish identity validation or domain ownership verification [Al-Zofi, 2025].

This architectural divergence exposes a clear socio-technical contradiction within corporate software engineering practices. Despite the established security advantages of Maven Central's verification model, enterprise software deployments overwhelmingly persist with npm and PyPI infrastructures [Al-Zofi, 2025]. Organizations accept these highly vulnerable, open attack surfaces due to institutional inertia, the significant sunk costs of existing dependency graphs, and the operational demand for rapid prototyping [Al-Zofi, 2025]. The massive scale of flat registries further exacerbates verification bottlenecks; for example, the npm ecosystem contains nearly 3.4 million active packages, making comprehensive manual auditing or proactive malware scanning statistically and operationally unfeasible [Al-Zofi, 2025]. Historical analysis shows that when malicious packages are reported to flat registries, maintainers confirm and remove approximately 82% of the flagged dependencies, illustrating that these platforms remain highly reactive and dependent on post-facto community reporting [Duan et al., 2020].

Cross-language empirical evaluations explicitly map how these registry frameworks handle AI-generated dependencies. Under identical multi-model prompting evaluations, cross-language package hallucination frequencies result in 6,705 invalid dependency fabrications targeting the flat npm registry, whereas compiled, domain-verified ecosystems like Java's Maven Central record exactly 0 hallucination instances [Spracklen et al., 2025]. This resilience is achieved because the structural constraints of

Maven's group identifiers prevent the model from generating random, single-token names that map onto valid registration paths [Al-Zofi, 2025].

However, a pronounced gap in the technical literature is the failure to propose scalable, automated provenance controls that can be integrated directly into public flat registries without introducing severe publication latency or disrupting the open-source contribution lifecycle. Current architecture studies do not provide technical paths for flat namespaces to transition toward domain validation without breaking backward compatibility for millions of legacy components, leaving this as an unresolved structural challenge. Registry architecture directly influences the risks associated with AI-generated dependencies and package hallucinations.

2.4.4 Software Supply Chain Risks in AI-generated Code

The emergence of automated coding systems drastically expands the enterprise supply chain risk surface by transitioning development workflows from human-monitored environments to autonomous code modification environments [Gandhi, 2025]. Traditional AI coding assistants operate under a reactive paradigm, requiring a human programmer to explicitly trigger prompts and manually review each code recommendation within the boundaries of a localized interface [Gandhi, 2025]. Conversely, modern agentic coding systems and vibe coding workflows—a development paradigm established in early 2025 where developers provide high-level intent while delegating executing, file writing, and execution rights to LLMs possess direct tool-use capabilities, multi-step reasoning loops, and environment interaction privileges [Gandhi, 2025]. This autonomy allows the model to act as a confused deputy; if an agent generates a hallucinated package dependency to satisfy a coding constraint, its autonomous action space enables it to execute shell commands and directly trigger package installations from public registries without human approval or authorization barriers [Al-Zofi, 2025].

Large-scale evidence shows that package fabrication rates are highly sensitive to model choice, language syntax, and parameter scaling. Closed-source, commercial models maintain a lower average dependency hallucination frequency of 5.2%, whereas open-

source architectures exhibit a significantly higher fabrication rate of 21.7% under matching prompt distributions [Al-Zofi, 2025]. Large-scale evaluations demonstrate that package hallucination frequencies across popular LLMs vary significantly, with commercial models showing lower rates compared to open-source implementations [Spracklen et al., 2025]. Empirical evidence shows that Python exhibits significantly higher package fabrication rates compared to JavaScript, while the Rust ecosystem displays distinct hallucination patterns with consistent rates across model families [Spracklen et al., 2025; Zheng et al., 2026].

To intercept these generative anomalies before they reach production pipelines, current research explores real-time hybrid architectures that combine inline static code analysis with AI-based semantic inspection directly within developer integrated development environments [Yadav et al., 2026]. However, documented failure modes reveal that these guardrails interact asymmetrically across model types and fail to capture systemic semantic manipulation [Qu et al., 2026]. While static analysis engines intercept up to 90.7% of explicit code injections, they are entirely incapable of identifying action-space hijacking achieved through workspace configuration files [Qu et al., 2026]. For example, an adversary can use indirect prompt injection to force an agent to generate a poisoned `pip.conf` or `.npmrc` file that quietly redirects all downstream pip or npm fetch requests to an attacker-controlled registry; because these configuration modifications represent legitimate file-write primitives, they completely bypass sandboxing constraints and safety alignment guardrails [Qu et al., 2026]. Furthermore, post-generation verification via model self-auditing is highly unreliable; prompting an LLM to self-verify its own code correctness yields an exceptionally low recall score of 0.02, rendering conversational self-correction ineffective against hidden supply chain defects [Woesle et al., 2025].

A critical methodological limitation across this mitigation literature is its heavy reliance on synthetic, single-file benchmarks like HumanEval or MBPP, which fail to replicate the architectural dependencies of multi-file enterprise repositories [Yang et al., 2026]. The reviewed texts do not provide any empirical evaluation regarding the processing latency, memory consumption, or computational overhead introduced by embedding real-time semantic inspection pipelines into continuous integration and continuous

deployment pipelines, leaving enterprise scalability as an completely unaddressed gap in current software engineering literature. Existing research demonstrates growing software supply chain risks associated with AI-generated code but remains fragmented, motivating the following section on mitigation strategies, detection techniques, and secure AI-assisted software development approaches.

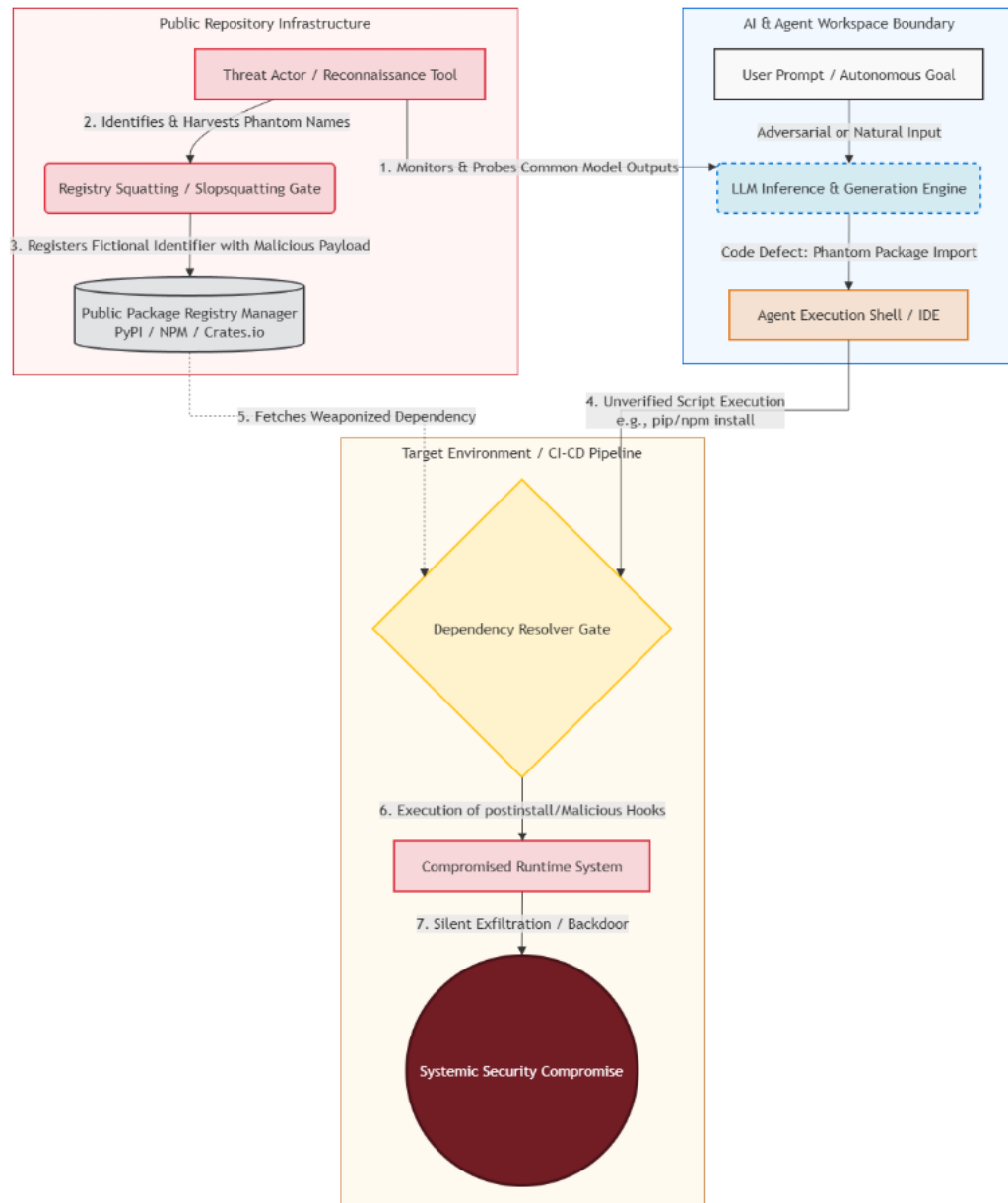


Figure 2-2 : Operational dataflow of AI-driven package hallucination propagation and slopsquatting execution within autonomous agent lifecycles.

2.5 Detection Approaches

2.5.1 Static Detection Methods

Static detection techniques identify hallucinated packages, malicious dependencies, and software supply chain anomalies within artificial intelligence assisted workflows before code execution or package installation occurs [Daoud, 2026]. These approaches encompass diverse methodologies, including package name validation, static dependency analysis, public registry verification, and rule-based or heuristic-based syntax scanning [Al-Zofi, 2025; Duan et al., 2020]. Traditional static analysis frameworks utilize predefined rule files and strict signature sets to isolate known structural defects or security weaknesses [Dubey and Madisetti, 2026]. In contrast, contemporary AI-aware static tools parse generated code sequences into Abstract Syntax Trees (ASTs) to cross-reference extracted symbols against an authoritative repository specification or an external API oracle [Tileria et al., 2026]. When evaluated against ungrounded conversational generation, registry validation methods provide early-stage perimeter defense by directly querying repository indices to verify if a recommended package identifier exists within the target ecosystem [Al-Zofi, 2025].

However, the literature highlights critical trade-offs and structural vulnerabilities across different registry validation techniques. Dependency graph analysis maps the cascading relationship between package coordinates to identify structural vulnerabilities or mismatched version constraints, whereas registry lookup approaches evaluate package names in isolation [Ohm and Stuke, 2023]. This distinction exposes a severe limitation when protecting flat namespaces such as Python's PyPI or JavaScript's npm registries, which lack the hierarchical namespace protections native to domain-verified environments like Java's Maven Central [Al-Zofi, 2025]. Large-scale empirical evaluations demonstrate that while static lookup filters successfully flag up to 94.8% of obvious package hallucinations, they are completely ineffective when facing sophisticated camouflage or zero-day slopsquatting attacks where an adversary has proactively registered an AI-invented identifier on a public registry before a developer initiates a build command [Al-Zofi, 2025].

A profound methodological gap in this domain relates to the scalability and continuous maintenance of static detection rulesets. Current tools rely heavily on static snapshots

of software ecosystems, meaning their fact-checking capability decays rapidly as public APIs and external library interfaces evolve over time [Jain et al., 2025]. Furthermore, tools like the Hallucination Inspector display severe performance regressions when evaluating complex asynchronous structures, underperforming significantly when analyzing runtime callbacks or when loose typing configurations force type under-approximation across the source codebase [Tileria et al., 2026]. The reviewed literature does not present automated mechanisms capable of dynamically synchronizing static rulesets against the non-deterministic vocabulary changes of large language models, leaving rule obsolescence as an unresolved engineering challenge. Conclude that while static techniques identify many dependency risks before execution, some threats only become observable during software execution, motivating runtime detection approaches.

2.5.2 Runtime Detection Methods

Runtime detection methods intercept malicious or hallucinated software dependencies while a program is actively running, evaluating live execution semantics rather than inert text patterns [Ohm and Stuke, 2023]. These dynamic methodologies involve sandboxing, dynamic dependency profiling, behavioral telemetry monitoring, and execution-based verification [Duan et al., 2020; Tian et al., 2025]. Traditional runtime security tools track low-level operating system telemetry such as file handles, network socket initialization, and system call frequencies to enforce absolute sandboxing barriers around execution environments [Ohm and Stuke, 2023]. Conversely, AI-aware runtime systems dynamically intercept program execution traces to validate functional behaviors directly against the semantic intent expressed in the original user prompt [Yang et al., 2026]. For example, the CodeHalu framework utilizes an execution-based verification engine that feeds inputs into generated code units and monitors data-flow compliance, effectively catching hidden mapping, naming, and resource consumption hallucinations that pass basic static syntax checks [Tian et al., 2025]. Similarly, the hybrid core of the SDHD architecture integrates a differential dynamic executor that validates structural execution paths to detect logical failures lacking regular syntactical markers [Yang et al., 2026].

Despite providing deeper visibility into software behavior, the deployment of runtime detection within enterprise environments introduces major operational friction and significant performance penalties. Sandboxing isolates untrusted dependencies in virtualized containers to protect host infrastructure, yet production monitoring must continuously profile running applications, introducing ongoing latency bottlenecks [Ohm and Stuke, 2023]. Dynamic profiling engines suffer from structurally restricted code coverage, as they remain completely incapable of identifying dormant malicious payloads nested in unexecuted logic paths or edge-case blocks that are not triggered by active test suites [Ohm and Stuke, 2023]. This vulnerability is explicitly weaponized against autonomous coding agents and vibe coding environments through action-space hijacking and stealthy supply chain injections [Qu et al., 2026]. Advanced adversarial setups utilize camouflage techniques like functional compliance disguise disguising unauthorized asset exfiltration as legitimate telemetry traffic or silent exception suppression to wrap malicious invocation hooks within try/except blocks, ensuring that abnormal traces are entirely invisible to standard runtime instrumentation layers [Qu et al., 2026].

A major research gap within this active body of literature is the failure to model the enterprise scalability, compute amplification, and operational latency of dynamic execution engines. Existing verification suites like CodeHalu or SDHD are built and tested exclusively using constrained, synthetic datasets (such as MBPP, HumanEval, or CodeHaluEval), which fail to mirror the multi-threaded dependencies, network configurations, and database interactions typical of industrial production systems [Tian et al., 2025; Yang et al., 2026]. The reviewed engineering texts provide no documentation regarding the resource overhead or performance degradation caused by maintaining continuous dynamic instrumentation layers within large-scale continuous integration pipelines, leaving the scalability of AI-aware dynamic detection unresolved. Conclude that neither static nor runtime detection alone provides complete protection, requiring a direct comparison of their capabilities and limitations.

2.5.3 Static vs Runtime Detection

A comparative synthesis of static and runtime approaches highlights a clear trade-off between early-stage prevention with limited semantic visibility and late-stage

behavioral observation accompanied by severe execution risk [Ohm and Stuke, 2023]. Static detection methods deliver ideal pipeline efficiency by intercepting defects at the pre-execution phase with zero computational risk to host environments, making them highly suitable for local integration into developer environments [Yadav et al., 2026]. However, static scanners are completely blind to dynamic code loading, obfuscated payloads, or configurations designed to manipulate environment variables at runtime [Qu et al., 2026]. Runtime approaches provide robust defense against zero-day dependency fabrications and active supply chain compromises, but they execute after package retrieval, meaning they only observe an anomaly after unverified components have already gained access to the system shell or build environment [Al-Zofi, 2025; Duan et al., 2020].

To bridge these defensive gaps, researchers propose unified hybrid architectures like the MALOSS framework, which combines metadata scanning, abstract syntax trees, and virtualized dynamic execution loops to detect repository anomalies [Duan et al., 2020]. When deployed across public package managers, this multi-layered approach achieved an 82% confirmation and removal rate across 339 intercepted malicious packages, highlighting the utility of multi-criteria analysis [Duan et al., 2020]. Nevertheless, acute tensions persist regarding optimal enterprise deployment models. Proponents of front-end static vetting emphasize low operational friction and rapid execution cycles [Dubey and Madiseti, 2026], whereas adversarial security evaluations demonstrate that up to 33.5% of sophisticated code-generation poisoning payloads easily bypass static rulesets, requiring dynamic semantic verification [Qu et al., 2026].

The empirical findings demonstrate a critical divergence in detection capability depending on the non-deterministic nature of the target attack vectors. The data highlights Qu et al. (2026) as a prominent outlier, where up to 33.5% of sophisticated contextual payloads completely bypassed state-of-the-art static layers, a trend confirming that lexical checks fail against intent-level manipulations. This fragmentation underscores that single-layer tools remain inadequate, while hybrid platforms scale poorly due to severe cross-language compatibility barriers and high computational resource footprints.

A primary open challenge restricting the field is the absence of comprehensive cross-ecosystem benchmarks capable of standardizing static and dynamic threat metrics uniformly across multiple programming languages. Current defense frameworks are heavily partitioned by ecosystem boundaries; for example, the hybrid SDHD framework is entirely restricted to Python environments [Yang et al., 2026], while the Hallucination Inspector is tailored specifically for Android API migrations [Tileria et al., 2026]. Consequently, current literature lacks unified automated response architectures or real-time validation layers capable of defending autonomous agent environments or multi-language repository structures without introducing severe execution delays, establishing a clear path for future security research. Existing detection approaches remain fragmented and exhibit complementary strengths and weaknesses, providing the rationale for the following section on mitigation strategies, detection techniques, and secure AI-assisted software development approaches.

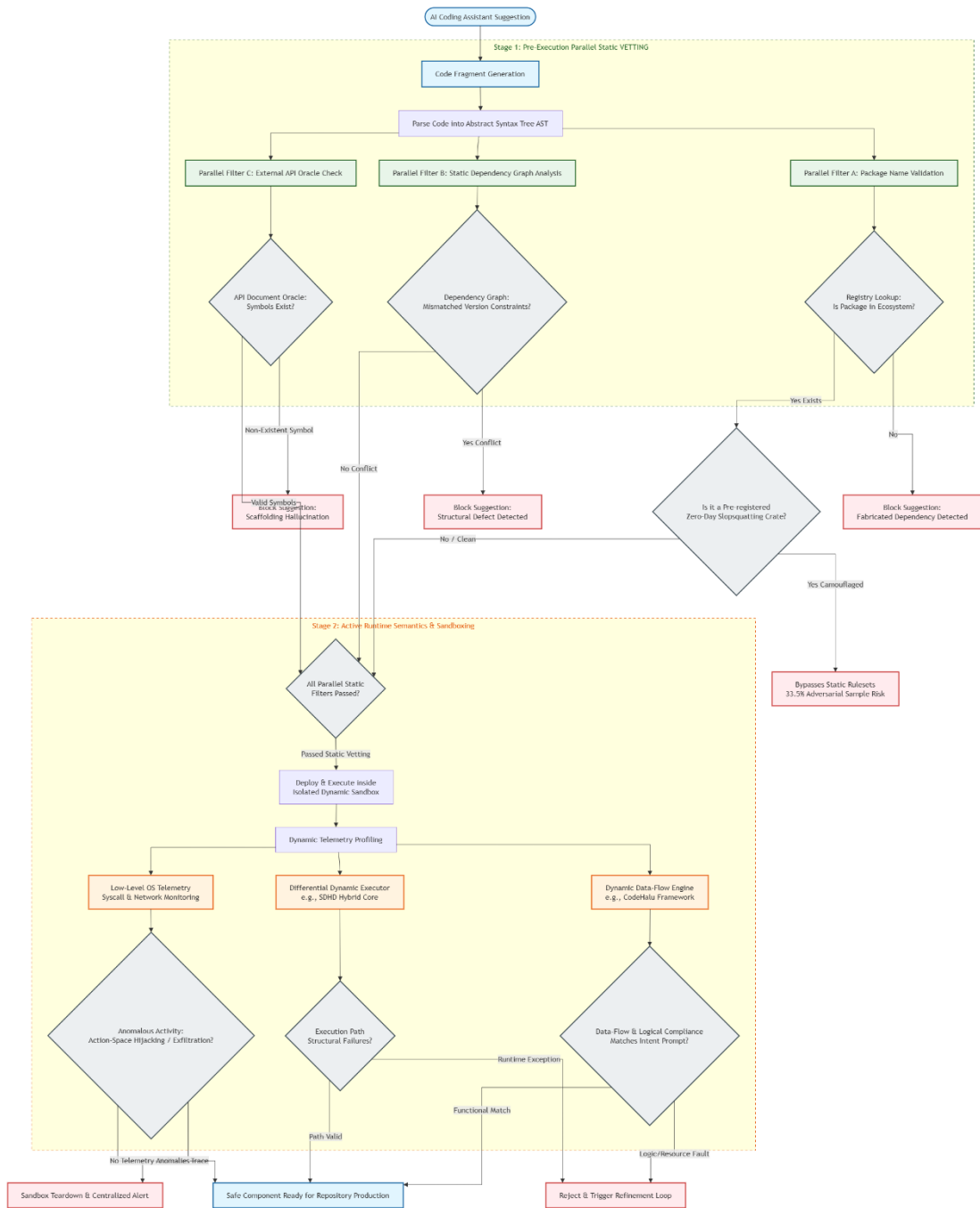


Figure 2-3 : details a multi-stage defensive architecture

2.6 Hallucination Mitigation Strategies

2.6.1 Retrieval-Augmented Generation (RAG)

A primary division in automated mitigation literature contrasts standalone model-only code generation against retrieval-augmented generation architectures. Standalone models rely entirely on their parametric memory, making them highly vulnerable to

strict temporal cut-offs and unable to realign with evolving package manager indexes [Twist et al., 2026]. To minimize package fabrications, RAG frameworks introduce external knowledge sources to bound model predictions at generation time [Spracklen et al., 2025]. However, a severe limitation arises when RAG platforms inherit standard natural language processing techniques, relying on chunking-based text indexing and semantic similarity searches across code corpora [Le-Anh et al., 2026]. Chunking-based indexing fragments functional code syntax, destroying the coherence between adjacent blocks and overlooking critical structural context [Le-Anh et al., 2026]. Furthermore, pure similarity-driven retrievers systematically miss non-contiguous but functionally relevant repository dependencies, such as cross-file helper functions, localized classes, or global variable definitions, reducing accuracy in complex development tasks [Le-Anh et al., 2026]. To resolve this technical barrier, contemporary code-specific frameworks implement structure-aware indexing strategies that represent multi-file repositories as hierarchical syntax trees, utilizing dependency-aware retrievers to isolate true programming dependencies over simple text matches on challenging repository benchmarks like DevEval and RepoExec [Le-Anh et al., 2026].

A significant tension exists in the literature regarding the consistency of RAG-driven hallucination suppression across different language boundaries and parameters. While large-scale evaluations across 576,000 code samples demonstrate that vector lookups constrain package fabrications by enforcing registry existence boundaries [Spracklen et al., 2025], evidence from compiled environments indicates that RAG provides only modest improvements when mitigating crate hallucinations in Rust code generation [Zheng et al., 2026]. This suggests that simple context expansion remains insufficient for establishing secure code environments without deeper model alignment loops [Zheng et al., 2026]. Moreover, empirical evidence confirms that sub-optimal document selection can actively introduce context confusion into model reasoning loops; when a retriever feeds irrelevant or mismatched context slices into a model, it triggers a severe 39.02% absolute drop in functional correctness for high-frequency, well-known APIs, transforming an intended mitigation into a source of generative distraction [Jain et al., 2025].

This behavior highlights a persistent research gap in the current literature. While retrieval approaches are widely proposed as a cost-effective alternative to the prohibitive financial and computational costs of continuous model fine-tuning and training updates [Spracklen et al., 2025], the reviewed studies fail to address or quantify the operational latency overhead, execution bottlenecks, or memory consumption patterns introduced by running live textual retrieval pipelines within industrial enterprise continuous integration and continuous deployment environments. Conclude that while static techniques identify many dependency risks before execution, some threats only become observable during software execution, motivating runtime detection approaches.

2.6.2 Knowledge Graph Grounding

Knowledge graph-based grounding approaches address the syntax limitations of unstructured document retrieval by modeling software registries and code environments as rigid, relational networks [Li et al., 2026]. Rather than parsing reference material into isolated vector chunks, structured representations natively preserve the explicit relationships connecting software entities, package version strings, and specific third-party API configurations [Li et al., 2026; Liu et al., 2025b]. Defensive frameworks, such as StepwiseGraphFix, execute knowledge graph reasoning and iterative feedback loops to progressively intercept API misuses in code targeting unfamiliar libraries, delivering an 8.1% relative improvement in repair success rates and a 4.0% relative improvement in test pass scores across bimodal datasets like UnifiedFixEval [Li et al., 2026]. Similarly, structured knowledge injection methods that combine pre-execution database context enrichment with post-execution heuristic error optimization achieve massive reliability gains, reducing program error rates from an ungrounded baseline of 61.18% down to 16.64% for commercial models [Liu et al., 2025b].

However, a fundamental architectural contradiction manifests when balancing the explainability of structured knowledge graphs against the practical challenges of graph maintenance. While relational schemas deliver excellent query efficiency and highly interpretable error prioritization logs [Dubey and Madiseti, 2026], their defensive accuracy is entirely dependent on graph completeness and dataset fidelity [Li et al.,

2026]. Documented failure modes reveal that when an underlying graph is incomplete or incorporates outdated relationship paths from unvetted private repositories, the graph reasoning subsystem introduces secondary hallucination vectors, actively guiding the model to generate logic errors or misuse interfaces based on faulty relationship graphs [Li et al., 2026; Liu et al., 2025b].

This limitation exposes a widespread research gap in current technical literature. The current state-of-the-art remains restricted to offline, closed-loop experiments on predefined libraries, and the existing texts provide no automated technical architectures, dynamic synchronization frameworks, or scalability modeling capable of expanding knowledge graph updates across millions of rapidly mutating open-source libraries without manual developer annotation. The operational maintenance cost of tracking package dependencies at ecosystem scale remains completely unexamined. Highlight that although knowledge graphs provide structured understanding, developers often rely directly on official API documentation, creating another grounding approach.

2.6.3 API Documentation Grounding

API documentation grounding improves code generation reliability by embedding official interface specifications, parameter rules, and package method signatures directly into the generation prompt context [Jain et al., 2025; Washio and Miyao, 2022]. This approach directly combats the long-tail library problem, where models possess sparse internal parametric representations of newly released or rare libraries, forcing the model's copy mechanisms to extract valid code signatures from the official document set rather than inventing nonexistent entities [Washio and Miyao, 2022]. Quantitative evaluations on the CloudAPIBench dataset demonstrate that documentation-augmented generation (DAG) pipelines provide substantial protection for low-frequency APIs, elevating the valid cloud invocation rate of commercial foundation models like GPT-4o from a baseline score of 38.58% up to 47.94% [Jain et al., 2025].

Nevertheless, the implementation of official documentation grounding uncovers an asymmetric trade-off between semantic precision and the physical limits of attention context windows. Although reading documentation sets allows foundation models to

navigate unseen library spaces like human programmers without additional training [Washio and Miyao, 2022], the technique frequently degrades when applied to complex, repository-level enterprise architectures . Outdated documentation snapshots or incomplete signature descriptions routinely cause compilation failures because models fail to resolve contextual conflicts when parametric knowledge actively contradicts the provided text prompt [Jain et al., 2025].

This reveals an immediate contextual research gap: current literature does not explore the technical complexity of automated doc-string extraction, nor does it provide empirical usability metrics detailing the manual preparation effort and developer friction required to continually maintain and prune internal corporate documentation indexes to prevent context window saturation during live engineering loops. Conclude that different grounding strategies address hallucination from different perspectives, requiring a comparative evaluation of their practical trade-offs.

2.6.4 Comparative Analysis: RAG vs Knowledge Graph vs API Documentation

A comparative synthesis of automated mitigation strategies establishes that while retrieval-augmented generation, knowledge graph reasoning, and API documentation pipelines provide essential perimeters for defending software supply chains, they present highly divergent execution footprints and technical limitations [Li et al., 2026; Jain et al., 2025]. Unstructured retrieval models scale efficiently across broad documentation repositories with minimal initial creation cost [Spracklen et al., 2025], but their reliance on similarity matching renders them vulnerable to retriever poisoning and high detection bypass rates when confronted with adversarial context manipulation or stealthy code injections [Le-Anh et al., 2026; Qu et al., 2026]. Structured knowledge graph systems deliver unparalleled dependency mapping and advanced structural reasoning, but they remain heavily bottlenecked by model context window constraints and introduce intense local infrastructure costs [Li et al., 2026; Liu et al., 2025b]. API documentation pipelines provide a highly accurate mechanism for expanding long-tail API correctness [Washio and Miyao, 2022], but their defensive efficacy is completely dependent on document freshness, mapping accuracy, and they fail when type under-

approximation limits symbol visibility across the repository [Jain et al., 2025; Tileria et al., 2026].

To overcome the architectural failures of standalone methods, contemporary frameworks explore unified hybrid platforms that tie live repository index checks directly into continuous validation workflows. Proactive development environments that integrate real-time Model Context Protocol (MCP) checking servers to cross-reference candidates against public registry indices demonstrate the lowest rate of package hallucination slip-throughs.

To demonstrate how these distinct validation pillars operate in tandem, Figure 2.4 outlines a unified multi-layered hybrid architecture. The schematic tracks an inbound query as it branches concurrently through an unstructured vector RAG/DAG pipeline and a structured knowledge graph relational schema match, consolidating context boundaries before code generation and running post-execution loops to capture latent anomalies.

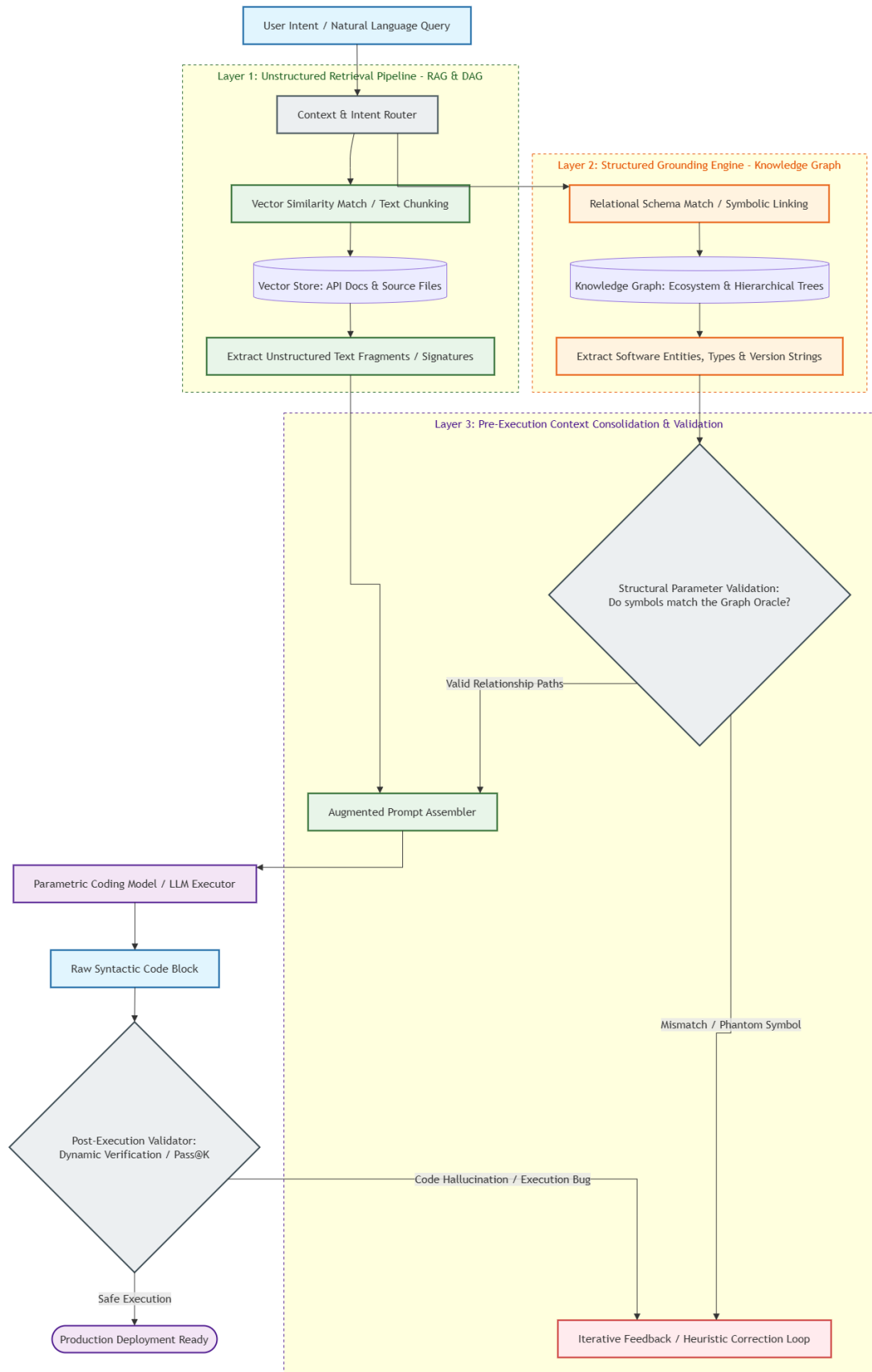


Figure 2-4 : Operational layout of a multi-layered hybrid mitigation pipeline combining unstructured text vectors, structured knowledge graph oracles, and post-execution runtime validation.

However, these combined mitigation frameworks introduce extreme latency penalties and are strictly throttled by the external API rate limits of target package registries, causing frequent build timeouts when deployed across nested industrial projects [Li et al., 2026].

Table 2-2 : Comparative Evaluation of Hallucination Mitigation Strategies

Aspect	Retrieval-Augmented Generation (RAG)	Knowledge Graph Grounding	API Documentation Grounding
Accuracy	Moderate; limited by semantic similarity limits and chunk coherence [Le-Anh et al., 2026]	High; driven by explicit type hierarchies and relational bounds [Li et al., 2026]	High for long-tail targets; prone to degradation on common paths [Jain et al., 2025]
Latency	Low to moderate pre-execution vector lookup latency [Le-Anh et al., 2026]	High runtime latency due to dynamic feedback loops and multi-step reasoning [Li et al., 2026]	Low text insertion latency; bounded by the document retriever speed [Jain et al., 2025]
Cost	Low implementation cost; avoids explicit architecture adaptations [Spracklen et al., 2025]	High infrastructure costs for graph schema construction [Liu et al., 2025b]	Low processing costs; bounded by basic text formatting steps [Washio and Miyao, 2022]
Hallucination reduction	Restricts package fabrication by supplying live repository indexing boundaries [Spracklen et al., 2025]	Reduces API error rates from 61.18% to 16.64% via structured knowledge injection [Liu et al., 2025b]	Increases valid invocation rates of low-frequency cloud APIs from 38.58% to 47.94% [Jain et al., 2025]
Scalability	Scales efficiently across huge textual corpora; ignores	Strictly constrained by model context window capacities	Bounded by text space; fails on massive enterprise multi-file

	structural code bounds [Le-Anh et al., 2026]	and graph node counts [Li et al., 2026]	structures [Washio and Miyao, 2022]
Maintenance	Requires continuous updating of textual index databases to avoid factual lag [Spracklen et al., 2025]	Intense manual effort; demands schema re-generation for all API modifications [Liu et al., 2025b]	Requires ongoing documentation indexing updates and keyword pruning [Jain et al., 2025]
Enterprise suitability	Bounded by text-chunk errors and vector retriever poisoning risks [Qu et al., 2026]	Delivers robust explainability logs; limited by private code coverage gaps [Li et al., 2026]	Bounded by manual formatting effort and nested codebase complexity [Washio and Miyao, 2022]
Weaknesses	Loses syntax coherence between code blocks; misses remote dependencies [Le-Anh et al., 2026]	Incomplete schemas introduce secondary hallucination loops into reasoning layers [Liu et al., 2025b]	Sub-optimal retrievers cause an absolute 39.02% pass rate drop on well-known interfaces [Jain et al., 2025]

The empirical trends compiled in Table 2.2 establish that while structured graph configurations deliver the highest baseline precision for intercepting method defects, their implementation costs remain prohibitively high for dynamic software environments. The data highlights a critical vulnerability in unstructured retrieval methods, where sub-optimal retriever matching introduces an absolute 39.02% performance degradation on mainstream programming tasks [Jain et al., 2025]. This pattern reinforces that unguided context injection underconditions model generation, requiring the adoption of precise, language-aware boundary controls over simple text-matching solutions.

A cross-cutting research challenge restricting this field is the complete absence of a standardized, multi-registry evaluation framework capable of measuring mitigation accuracy across flat namespaces and scoped domains simultaneously under identical parameter sets. Consequently, long-term organizational maintenance costs, resource amplification profiles, and the precise rate of developer trust degradation over extended

exposure to grounded-but-flawed code recommendations remain completely unmapped, establishing a clear avenue for future empirical research. Conclude that current mitigation strategies reduce hallucination risks but involve significant trade-offs, motivating further investigation into comprehensive detection, validation, and secure AI-assisted software development frameworks.

2.7 Evaluation of Hallucination Mitigation

2.7.1 Comparative Evaluation of Hallucination Mitigation Studies

The architectural validation of hallucination suppression techniques within automated code intelligence workflows defines an active area of empirical contestation. Because source code generation demands deterministic operational compliance alongside absolute syntactic alignment, historical evaluation approaches that relied entirely on token-level or match-based text similarity metrics have collapsed under high false-negative rates [Lian et al., 2024]. Contemporary evaluation studies focus on cross-examining grounding paradigms, directly contrasting unstructured Retrieval-Augmented Generation (RAG) against relational Knowledge Graph reasoning and official API documentation injection [Jain et al., 2025; Li et al., 2026; Spracklen et al., 2025]. Quantitative analysis of these interventions demonstrates a severe performance tension; the reported effectiveness of any given mitigation framework is fundamentally dependent on model scaling boundaries, prompt composition style, and ecosystem-specific resolution traits rather than model capability alone [Tian et al., 2025; Twist et al., 2026].

A primary division in automated mitigation literature contrasts unstructured text retrieval models against structured knowledge representations. Unstructured text retrieval platforms scale efficiently across broad documentation repositories with minimal initial creation cost [Spracklen et al., 2025], but their reliance on semantic similarity matching renders them highly vulnerable to context fragmenting and retriever poisoning when confronted with adversarial context manipulation [Le-Anh et al., 2026; Qu et al., 2026]. Standard text chunking slices functional code block sequences

arbitrarily, destroying syntactic coherence and routinely missing non-contiguous but functionally vital repository dependencies across multi-file architectures [Le-Anh et al., 2026]. Structured knowledge graph systems solve these syntax chunking errors by encoding library type hierarchies and data-flow constraints natively [Li et al., 2026]. Frameworks executing graph reasoning achieve substantial reliability improvements reducing program error rates from an ungrounded baseline of 61.18% down to 16.64% [Liu et al., 2025b] but their deployment practicality remains heavily bottlenecked by intense graph schema construction costs and context window capacity boundaries [Li et al., 2026; Liu et al., 2025b]. This introduces an explicit research gap, as current literature fails to provide dynamic synchronization technical paths to update knowledge graph nodes against millions of rapidly mutating open-source dependencies without manual developer annotation.

Official API documentation grounding presents a parallel mitigation approach, inserting interface specifications and package method signatures directly into the generation context to expand long-tail symbol correctness [Jain et al., 2025; Washio and Miyao, 2022]. Quantitative evaluations on CloudAPIBench demonstrate that documentation-augmented generation (DAG) lifts valid cloud invocation rates for low-frequency APIs up to 47.94% [Jain et al., 2025]. However, documented failure modes reveal that sub-optimal document selection can actively cause context dilution, introducing a severe 39.02% absolute drop in model correctness on high-frequency, well-known interfaces due to retriever noise injection [Jain et al., 2025]. Furthermore, the technical texts do not provide empirical metrics detailing the manual preparation effort or the continuous processing latency introduced by embedding real-time semantic inspection pipelines into continuous enterprise deployment environments, leaving operational scalability unaddressed.

The generalizability of mitigation frameworks is further polarized by the architectural dichotomy between commercial closed-source networks and open-source specialized coding model families [Spracklen et al., 2025]. Proprietary systems like GPT-4o show strong contextual resilience when augmented with documentation filters [Jain et al., 2025], yet their closed weights prevent independent scientific verification of their internal factual adjustments [Liu et al., 2026]. Open-source model families including

DeepSeek-Coder, Qwen-Coder, Llama, and CodeLlama variants offer complete parameter transparency but exhibit high performance degradation under context fluctuations and adversarial prompt slices [2025; Zheng et al., 2026]. For example, package hallucination propensities vary significantly across configurations, with commercial architectures tracking a lower fabrication rate of 5.2% compared to a 21.7% fabrication frequency across open-source models [Al-Zofi, 2025]. While structural constraints in prompt text minimize dependency errors, open-source architectures display steep parameter-dependent variance, where smaller parameter setups suffer severe error spikes [Spracklen et al., 2025]. This performance gap reflects the extreme sensitivity of open-source weights to sampling temperature parameters, where increased randomness causes package fabrication frequencies to scale sharply higher [Spracklen et al., 2025]. Longitudinal datasets assessing how these architectural variations affect the long-term decay of developer trust under prolonged industrial exposure are completely absent from the reviewed literature.

A final tension within the empirical evidence stems from fragmented evaluation protocols and benchmark selections [Daoud, 2026]. Traditional studies evaluated prompting enhancements on functional snippet datasets like HumanEval and MBPP using match-based metrics [Lian et al., 2024]. However, match-based evaluation methods fail to accommodate valid implementation variability, causing exceptionally high false-negative rates by misclassifying up to 43.66% of functionally correct generative code changes [Lian et al., 2024]. To overcome this blind spot, contemporary methodologies utilize execution-based verification frameworks that feed inputs into generated code blocks and monitor data-flow compliance via dynamic test cases [Tian et al., 2025; Yang et al., 2026]. Dynamic execution algorithms like Code-Halu expose hidden logic and mapping hallucinations that pass basic static syntax filters [Tian et al., 2025], but their testing environments remain restricted to synthetic snippets. Iterative refinement models and self-correction prompting loops report high success rates on functional micro-benchmarks but introduce extreme computational overhead, often reducing overall correctness on real-world workloads due to token-space limits [Liu et al., 2026; Zheng et al., 2026]. Moreover, micro-benchmarks targeting temporal boundaries reveal extreme library fabrication rates up to 85% under time-based queries

[Twist et al., 2026], a critical security threat surface that traditional functional correctness benchmarks entirely fail to capture.

The empirical distributions reflect an asynchronous pattern where mitigation success rates correspond directly to the level of contextual alignment enforced during the generation loop. Frameworks relying on loose prompt tuning or unstructured retrieval display significant susceptibility to high temperature configurations, with library fabrications spiking to an 85% extreme threshold under temporal constraint requests [Twist et al., 2026]. This clear baseline variance underscores a structural fragmentation across current verification protocols, demonstrating that text-level checks fail to guarantee security boundaries without compiling live registry validations or execution environments.

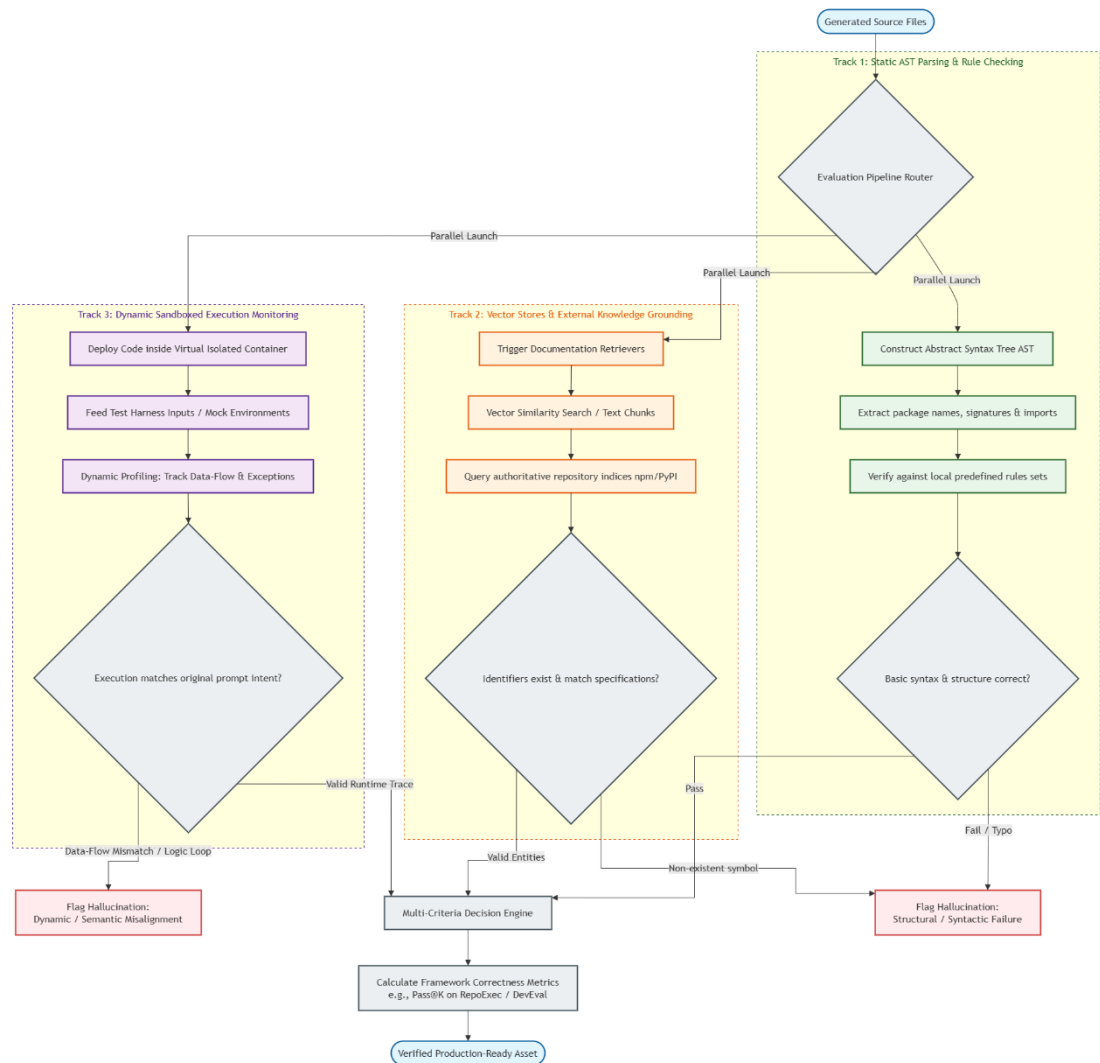


Figure 2-5 : Automated evaluation framework mapping parallel static checking, database grounding lookup, and sandboxed execution telemetry validation.

Unresolved gaps regarding cross-registry synchronization bounds and real-world developer alignment thresholds continue to limit the generalizability of these techniques. Consequently, the fragmented nature of these empirical results remains inconsistent across distinct programming environments, datasets, and execution platforms, providing the structural rationale and technical motivation for the comprehensive, multi-layered validation framework presented in the following chapter.

2.8 Research Gaps

2.8.1 Synthesis of Research Gaps Across the Literature

The thematic evaluation of automated software engineering literature reveals a critical gap between laboratory-based performance optimizations and the unstructured realities of operational software ecosystems. Current evaluation methodologies predominantly rely on standalone micro-benchmarks that fail to replicate the structural complexity of multi-file enterprise architectures [Le-Anh et al., 2026; Lian et al., 2024; Tian et al., 2025; Tripathi et al., 2025]. Within these synthetic environments, traditional text-matching metrics yield massive false-negative rates [Lian et al., 2024], while execution-based verifications remain confined to isolated snippets rather than multi-layered corporate repositories [Tian et al., 2025; Yang et al., 2026]. Furthermore, there is a distinct absence of longitudinal empirical data tracking developer cognitive fatigue and the decay of trust when engineers are exposed to repetitive hallucinated outputs.

This testing limitation severely restricts the understanding of emerging software supply chain attack surfaces, particularly AI-induced "slopsquatting" [Al-Zofi, 2025]. While studies show that flat-namespace registries (e.g., PyPI, npm) are highly susceptible to package fabrications compared to domain-verified ecosystems like Maven Central [Al-Zofi, 2025; Spracklen et al., 2025], current literature lacks empirical telemetry on live, real-world adversary publication patterns. This vulnerability escalates critically within modern agentic coding environments. As generative models transition from passive, human-in-the-loop assistants into autonomous actors with system-level execution

privileges [Al-Zofi, 2025; Gandhi, 2025], they become highly susceptible to context poisoning and action-space hijacking [Qu et al., 2026; Yadav et al., 2026]. Despite this proven susceptibility, comparative safety benchmarks evaluating the operational boundaries and installation error distributions of leading agentic engines remain undefined.

Finally, existing defense and mitigation architectures introduce severe system bottlenecks and unresolved operational trade-offs. Static and dynamic detection layers struggle with type under-approximation and frequently miss dormant malicious payloads disguised by functional camouflage [Qu et al., 2026; Tian et al., 2025; Tileria et al., 2026; Yang et al., 2026]. Simultaneously, context grounding strategies—such as retrieval-augmented generation, knowledge graphs, and API document injection—suffer from context dilution, semantic fragmentation, and intense schema construction penalties [Jain et al., 2025; Le-Anh et al., 2026; Li et al., 2026; Liu et al., 2025b; Spracklen et al., 2025]. Efforts to integrate real-time validation into continuous integration loops are actively throttled by external API fetch limits [Li et al., 2026; Zhao et al., 2025]. Ultimately, because existing paradigms are heavily partitioned by language boundaries, reliant on static snapshots, and unverified against high-privilege autonomous agents, they fail to guarantee factual accuracy [Twist et al., 2026] or adequately secure next-generation engineering pipelines.

3. Methodology

3.1 Introduction and Methodological Philosophy

This chapter details the methodological procedures employed to systematically investigate and quantify the security risks introduced by artificial intelligence code generation tools. Specifically, it establishes the framework used to assess the attack surface created by hallucinated application programming interfaces and software dependencies. The methodology is structured to ensure that the evaluation of autonomous coding environments and their subsequent vulnerabilities are measured with high empirical rigor and reproducibility.

3.1.1 Research Philosophy and Approach

This study adopts a positivist research philosophy, grounded in the premise that the phenomenon of code hallucination and its resulting supply chain vulnerabilities can be objectively observed, measured, and quantified independent of researcher bias. The positivist paradigm is highly appropriate for cybersecurity and software engineering research where the objective is to determine the exact frequencies of deterministic failure modes within large language models [Gao et al., 2025].

Consistent with positivism, the research employs a deductive approach. The investigation begins with established theoretical constructs regarding large language model reliability and software supply chain threat models to formulate specific, testable hypotheses. These hypotheses predict relationships between the operationalized variables, such as the expected differences in vulnerability introduction rates between varying levels of artificial intelligence autonomy. Consequently, a quantitative methodological posture is maintained throughout the study. The nature of the defined variables necessitates strict numerical and categorical measurement. For example, the primary dependent variable, defined as the prevalence of vulnerable dependencies, is measured through the absolute count of non-existent packages successfully imported in the generated code [Spracklen et al., 2025]. Qualitative interpretations are insufficient for evaluating systemic architectural flaws or exploitability rates; therefore, the study relies on statistical significance,

programmatic validation of dependency existence, and standardized experimental test metrics to confirm or reject the proposed hypotheses [Spracklen et al., 2025].

3.1.2 Research Design Overview

The research utilizes a multi-phase quantitative experimental design to address the core objectives of the study. This structured approach allows for the controlled manipulation and observation of the variables defined in the conceptual framework. The design is built around the manipulation of the primary independent variable, the workflow type, allowing for direct experimental comparison between traditional autocomplete configurations and autonomous agentic coding workflows [Gandhi, 2025]. By exposing these different execution environments to standardized coding tasks, the experimental framework systematically captures the dependent variable, recording the precise frequency of generated vulnerable dependencies across multiple iterations.

To address the contextual factors influencing this attack surface, the research design integrates specific moderating variables. It tests the generated outputs against different registry architectures, comparing the exploitation potential within first-come-first-served flat namespaces against domain-verified namespaces [Duan et al., 2020]. Additionally, the design incorporates structured survey instruments to capture developer expertise as a second moderating variable. This captures how years of professional experience and familiarity with supply chain threats influence the end-user acceptance of hallucinated code [Liu et al., 2026].

Finally, the research design incorporates an evaluation of a mediating variable, specifically the application of a verification and mitigation layer. The experimental testbed programmatically applies static and dynamic analysis mechanisms to the generated code to determine if vulnerable dependencies can be successfully intercepted prior to execution [Yang et al., 2026].

The remainder of this chapter is structured to provide detailed operational procedures for this design. It outlines the specific data collection methods, the automated instrumentation used for programmatic dependency extraction, the sampling strategies

for both the code generation prompts and human participants, and the statistical techniques selected for formal hypothesis testing.

3.3 Study Design and Research Framework

3.3.1 Overview of the Multi-Phase Research Design

This study adopts a positivist, quantitative experimental design structured across four sequential phases to systematically evaluate the attack surface introduced by artificial intelligence code generation tools. The first phase, Dataset Creation, systematically varies the independent variables by generating code snippets across different model architectures, workflow types, programming environments, functional categories, and prompt structures. The second phase, Dependency Extraction and Validation, utilizes automated parsing instrumentation to isolate all external libraries recommended within the generated code and programmatically queries official package registries to verify their existence.

The third phase, Risk Assessment, calculates the dependent variables by quantifying the absolute frequency of hallucinated dependencies, determining the immediate exploitability of the fabricated namespaces, and assigning a composite severity score to each identified vulnerability. The final phase, Mitigation Framework Development, assesses the efficacy of mediating verification layers, applying static and dynamic analysis mechanisms to determine whether the fabricated dependencies can be intercepted prior to execution. This multi-phase approach ensures that the measurement of vulnerabilities remains objective, reproducible, and isolated from subjective human evaluation frameworks.

3.3.2 Conceptual Framework

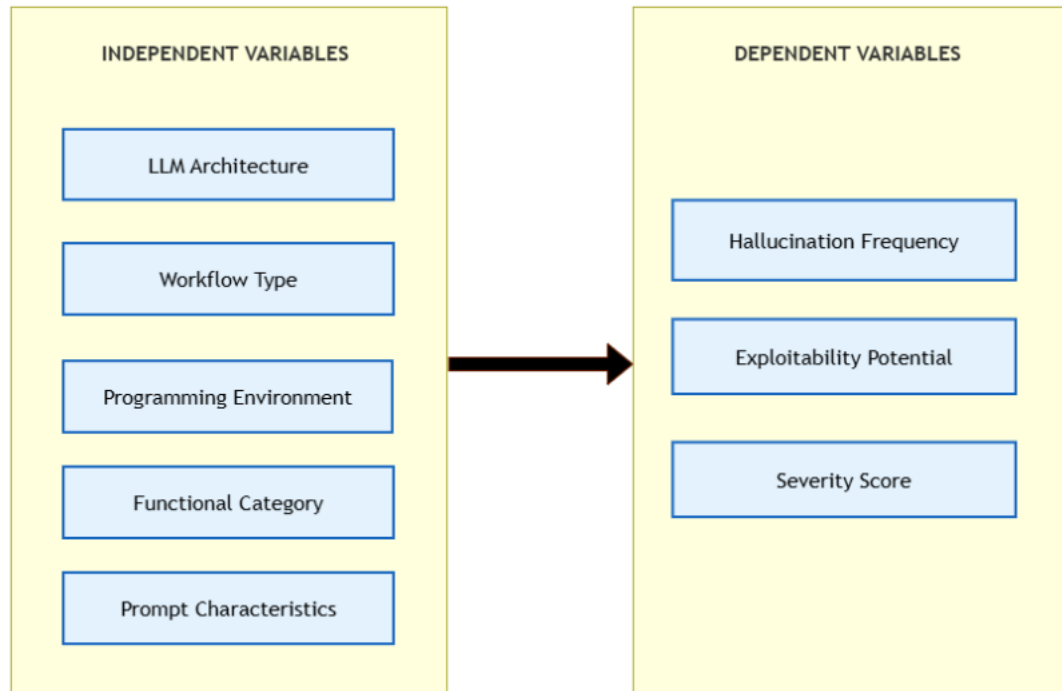


Figure 3-0-1 : Conceptual Framework Diagram.

The conceptual framework illustrates the precise architectural interactions between generative artificial intelligence systems, operational environments, and the resulting software supply chain vulnerabilities. It provides a structured mapping of how manipulated inputs and contextual factors influence the manifestation and severity of fabricated software dependencies. To measure these interactions empirically, the framework relies on a specific set of independent and dependent variables.

Independent Variables

- **LLM Architecture:** This categorical variable defines the underlying structural and parametric design of the foundation model utilized for code generation. It distinguishes between proprietary, closed-weight models and open-source alternatives, measuring how variations in model size and training data influence output reliability.
- **Workflow Type:** This categorical variable measures the level of artificial intelligence autonomy within the development pipeline. It contrasts traditional,

reactive autocomplete mechanisms against autonomous agentic workflows that possess file-system access and package execution privileges.

- **Programming Environment:** This categorical variable defines the language ecosystem and its corresponding repository infrastructure. It assesses the structural differences between first-come-first-served flat namespaces, such as npm or PyPI, and domain-verified infrastructures like Maven Central .
- **Functional Category:** This categorical variable classifies the intended objective of the required code snippet. It differentiates between routine data manipulation tasks and complex, security-critical implementations, measuring how domain complexity affects the generative stability of the model.
- **Prompt Characteristics:** This variable measures the specific variations introduced during the input stage, including the degree of user error, language ambiguity, or explicit constraints provided in the developer query.

Dependent Variables

- **Hallucination Frequency:** This quantitative variable measures the absolute count and percentage rate of non-existent application programming interfaces and external packages generated by the artificial intelligence model during a standardized test run [Spracklen et al., 2025]..
- **Exploitability Potential:** This binary or ordinal variable measures the likelihood that a generated hallucination can be successfully weaponized. It evaluates whether the hallucinated package name conforms to the naming conventions of the target registry and whether the namespace is currently unregistered and available for adversarial claiming [Al-Zofi, 2025].
- **Severity Score:** This composite quantitative variable aggregates the impact of a successful dependency compromise. It calculates the overall risk based on the execution privileges of the workflow, the criticality of the targeted application, and the evasion capability of the hallucinated dependency against standard static analysis tools.

3.3.2.1 Theoretical Foundations

The conceptual framework integrates three established theoretical constructs to explain the mechanics of artificial intelligence-induced supply chain compromises. The first construct examines the reliability of artificial intelligence-generated content, focusing on the deterministic failure modes and semantic deviations inherent in large language models during structural code synthesis [Gao et al., 2025]. The second construct involves software supply chain trust models, emphasizing the implicit trust developers place in upstream dependency resolution mechanisms and the historical shift toward exploiting centralized package managers [Duan et al., 2020]. The third construct addresses the expanded attack surfaces resulting from artificial intelligence automation, observing how the integration of autonomous coding tools bypasses traditional human verification barriers, thereby creating novel pathways for vulnerability injection [Yadav et al., 2026].

3.3.2.2 Three-Domain Intersection Model

The research operationalizes the problem space through a Three-Domain Intersection Model, which isolates the exact vulnerability overlap evaluated in the experiment. The first domain represents the automated code generation workflows that introduce non-existent application programming interfaces. The second domain encapsulates the hallucinated dependencies themselves, constituting the vulnerable software artifacts. The third domain represents the software supply chain risk, dictated by specific registry architectures and execution environments. The physical intersection of these three domains represents the active attack surface where a generative model failure successfully translates into a deployable, exploitable vulnerability.

3.3.2.3 Three-Stage Threat Model

To map the operational lifecycle of this specific vulnerability, the methodology adopts a Three-Stage Threat Model. In the first stage, the artificial intelligence coding tool generates a code snippet containing a syntactically plausible but entirely fabricated dependency [Spracklen et al., 2025]. In the second stage, an attacker anticipating these model failures registers the hallucinated package name within a public repository,

executing an attack vector identified as slopsquatting [Al-Zofi, 2025]. In the third and final stage, the developer or the autonomous coding agent accepts the generated code and executes the installation command, thereby unknowingly downloading the malicious payload into the local environment.

3.3.3 Operationalization of Variables

To systematically measure the relationships proposed in the conceptual framework, the theoretical constructs of this study are operationalized into distinct, measurable variables. This deductive approach ensures that the evaluation of the artificial intelligence hallucination attack surface remains objective, quantifiable, and reproducible. The specific indicators, measurement scales, and experimental metrics for each variable translate high-level security concepts into empirical data points suitable for rigorous statistical analysis.

Table 3-1 : Operationalization of Research Variables

Variable	Indicators	Reference	Survey Question / Experimental Metric
Independent Variable (IV): Workflow Type (A)	Categorical: 1 = Traditional Autocomplete (e.g., GitHub Copilot); 2 = Agentic / Vibe Coding workflow (e.g., Cursor, OpenHands).	[Gandhi, 2025], [Park, 2025]	<i>Experimental Test Metric:</i> Which specific AI coding workflow/assistant is being utilized for the generation of the target code snippet during this test run?

Moderating Variable (ModV1): Developer Expertise (B)	Ordinal Scale: Composite score based on a pre-study survey assessing years of professional experience and familiarity with supply-chain threats (Novice, Intermediate, Advanced).	[Gandhi, 2025], [Liu et al., 2026]	<i>Survey Question:</i> "How many years of professional software development experience do you have, and how would you rate your familiarity with software supply-chain security threats on a scale of 1 to 5 (1 = Unfamiliar, 5 = Highly Expert)?"
Moderating Variable (ModV2): Registry Architecture (C)	Categorical: 1 = First-come-first-served flat namespace (e.g., npm, PyPI); 2 = Domain-verified namespace (e.g., Maven Central).	[Al-Zofi, 2025], [Duan et al., 2020]	<i>Experimental Test Metric:</i> Does the target programming language ecosystem utilize a flat registry model or a domain-verified registry model?
Mediating Variable (MedV): Verification & Mitigation Layer (Static/Dynamic Analysis) (D & E)	Binary/Categorical: Presence and type of verification applied (0 = None, 1 = Static Analysis, 2 = Dynamic/Runtime Execution validation).	[Yang et al., 2026], [Li et al., 2026]	<i>Experimental Test Metric:</i> Was the generated code successfully flagged by the active detection mechanism (Static or Dynamic) prior to package execution? (Yes/No)

Dependent Variable (DV): Prevalence of Vulnerable Dependencies	Quantitative Count: Total number of non-existent (hallucinated) packages successfully imported and recommended by the AI during a test run.	[Al-Zofi, 2025], [Spracklen et al., 2025], [Zheng et al., 2026]	<i>Experimental Test Metric:</i> What is the absolute count of hallucinated (fake) APIs/packages identified in the final AI-generated code output?
--	---	---	---

The primary independent variable of this study is the Workflow Type. It is operationalized as a categorical variable consisting of two distinct levels: traditional autocomplete workflows and autonomous agentic coding workflows [Gandhi, 2025]. The experimental metric directly records the specific artificial intelligence assistant utilized for the generation of the target code snippet during each test run. Treating this as a discrete categorical variable allows for the systematic isolation of model autonomy, enabling controlled experimental comparisons to determine if increased execution privileges significantly alter the rate of vulnerability generation.

To account for contextual factors that govern the ultimate success of a supply chain compromise, the research design integrates two moderating variables. The first moderating variable, Developer Expertise, is operationalized on an ordinal scale. This metric is captured via a composite score derived from a pre-study survey assessing the participant's years of professional software development experience and their self-reported familiarity with software supply chain security threats on a five-point scale [Gandhi, 2025]; [Liu et al., 2026]. This ordinal measurement is critical for quantifying human-in-the-loop variance, specifically measuring the degree to which advanced security awareness limits the acceptance of hallucinated artifacts.

The second moderating variable, Registry Architecture, evaluates the structural vulnerability of the target environment. It is measured as a categorical variable that contrasts first-come-first-served flat namespaces, such as npm or PyPI, against domain-verified namespaces, such as Maven Central [Al-Zofi, 2025]; [Duan et al.,

2020]. The experimental test metric records the specific registry model utilized by the target programming language ecosystem. This operationalization allows the study to statistically assess how distinct infrastructure policies moderate the exploitability of the generated packages.

To evaluate the efficacy of defensive countermeasures, the methodology includes a mediating variable defined as the Verification and Mitigation Layer. This is measured as a categorical variable indicating the presence and specific type of automated verification applied during the test run, defined as no verification, static analysis, or dynamic runtime execution validation [Li et al., 2026]; [Yang et al., 2026]. The corresponding experimental test metric records a binary outcome of whether the generated code was successfully flagged by the active detection mechanism prior to package execution. Measuring this variable allows the research to isolate the interception capability of the defense layer and its mediating effect on the final security posture.

Finally, the ultimate outcome of the experiment is captured by the dependent variable, defined as the Prevalence of Vulnerable Dependencies. This is operationalized strictly as a quantitative count, representing the absolute total number of non-existent, hallucinated application programming interfaces and packages successfully imported and recommended by the artificial intelligence model during a test run [Al-Zofi, 2025]; [Spracklen et al., 2025]; [Zheng et al., 2026]. Utilizing a strict integer count for the dependent variable eliminates subjective interpretation, providing a definitive, objective metric of the attack surface necessary for formal hypothesis testing.

3.3.4 Research Hypotheses

H1: Frequency Hypothesis

H1: The application of artificial intelligence code generation tools positively impacts 'Hallucination Frequency', producing hallucinated dependencies at a statistically significant rate greater than 5 percent of total generated dependencies.

H2: Environment-Specific Hypothesis

H2: The 'Programming Environment' impacts 'Hallucination Frequency', such that the Node.js/npm ecosystem exhibits higher hallucination frequencies than the Spring Boot/Maven ecosystem due to registry validation differences.

H3: Tool Consistency Hypothesis

H3: There is an association between 'LLM Architecture' and generated artifacts, such that different artificial intelligence tools hallucinate distinct package names with a limited overlap of less than 30 percent.

H4: Criticality Hypothesis

H4: The 'Functional Category' of the programming task impacts 'Hallucination Frequency', wherein security-critical functional categories generate more hallucinated dependencies than general-purpose categories.

H5: Exploitability Hypothesis

H5: 'Hallucination Frequency' positively impacts 'Exploitability Potential', wherein at least 60 percent of identified hallucinated dependencies represent immediately exploitable attack vectors.

3.4 Data Collection

The data collection phase of this research is designed to systematically generate, capture, and catalog the dependent variable, specifically the prevalence of vulnerable dependencies, across varying configurations of the independent and moderating variables. To maintain a positivist and highly reproducible experimental posture, the data collection relies on a strict, automated protocol rather than unstructured human interaction, ensuring that the measured hallucination frequencies are statistically robust and isolated from arbitrary prompt variance.

3.4.1 Programming Environments and Ecosystems

3.4.2 AI Tools and Models

The data collection utilizes a targeted selection of commercial and open-weight models to manipulate the primary independent variable, Workflow Type, encompassing both traditional autocomplete assistants and highly autonomous agentic execution engines.

ChatGPT (GPT-4 and GPT-3.5)

The OpenAI GPT-4 and GPT-3.5 models are utilized via their official application programming interfaces to serve as foundational baselines for general-purpose code generation. Because these models power a vast majority of web-based coding queries, capturing their output provides a necessary volume of generalized code data to measure baseline hallucination frequencies and establish the generic occurrence rate of non-existent packages [Spracklen et al., 2024].

GitHub Copilot

GitHub Copilot is selected to specifically represent the traditional autocomplete workflow type. As an integrated development environment plugin restricted primarily to active file context and inline suggestions, it represents the lower-autonomy spectrum of the independent variable. Evaluating Copilot is critical for capturing how isolated, reactive coding assistants hallucinate dependencies when prompted by inline comments or partial function declarations.

Google Gemini

The Google Gemini family of models is included to assess the performance of inherently multimodal architectures optimized for advanced reasoning. Gemini's integration into enterprise development suites makes it a critical target for evaluating enterprise-grade supply chain risks. Including this model ensures the dataset captures variations in how different proprietary training pipelines structure software dependency recommendations.

Anthropic Claude

The Anthropic Claude models, particularly the Claude 3.5 Sonnet variant, are selected due to their predominant integration into modern autonomous agentic workflows, such as Cursor and OpenHands [Gandhi, 2025]. Evaluating Claude provides the data

necessary to measure the higher-autonomy spectrum of the workflow type variable. This allows the study to assess the severe risks introduced when an artificial intelligence tool not only hallucinates a package but possesses the agentic execution privileges required to autonomously invoke package installation commands for that non-existent dependency.

3.4.3 Code Generation Protocol

3.4.3.1 Prompt Design and Categorization

To ensure the dependent variable is measured accurately without interference from inconsistent human phrasing, the research employs a structured prompt engineering protocol. Prompts are designed as deterministic, context-isolated inputs that mandate the inclusion of external libraries, forcing the models to execute dependency resolution logic.

3.4.3.2 Functional Categories

The code generation tasks are divided into six distinct functional categories relevant to modern backend software development. This categorization directly supports the testing of the criticality hypothesis, allowing for the statistical measurement of how domain complexity influences hallucination frequency.

Authentication and Authorization

Tasks involving identity verification, token parsing (e.g., JSON Web Tokens), and role-based access control implementations.

Database Connectivity and Integration

Tasks requiring object-relational mapping configuration, connection pooling, and execution of complex querying structures against relational or NoSQL databases.

File Handling and Processing

Tasks involving the ingestion, parsing, formatting, and structural validation of complex file types such as XML, CSV, and PDF documents.

API Development and Endpoints

Tasks focusing on exposing Representational State Transfer endpoints, managing asynchronous requests, and handling web-socket streaming.

Security Features and Encryption

Tasks requiring cryptographic implementations, hashing algorithms, secure random number generation, and payload sanitization.

Logging and Caching

Tasks focused on implementing telemetry, application performance monitoring, distributed caching layers, and structured output formatting.

3.4.3.3 Structured Prompt Templates

Each query utilizes a standardized template to enforce uniformity across the generated dataset. An example template takes the following structure: "You are an expert backend software engineer. Write a complete, production-ready implementation for [Functional Task] using [Target Ecosystem]. You must explicitly include all required package imports, dependency declarations, and configuration headers necessary to compile and execute this code. Do not use generic placeholders."

3.4.3.4 Prompt Execution and Response Capture

To execute the data collection, automated Python scripts interface directly with the respective model application programming interfaces. For graphical user interface-bound tools like GitHub Copilot, standardized test harnesses are used to simulate developer inputs within an isolated integrated development environment. The temperature parameter for all application programming interface calls is standardized (e.g., at 0.7) to allow for the natural generative variance inherent to these models while preserving overall reproducibility.

3.4.3.5 Metadata Recording

Upon the completion of a generation cycle, the response is captured and logged into the dataset alongside a strict set of metadata fields mapped directly to the study's operationalization parameters. The recorded fields include the Prompt Identifier, Target Ecosystem (Registry Architecture), Functional Category, Tool Name and

Version (Workflow Type), Generation Timestamp, the Raw Code Output, and an array of Extracted Package Names.

3.4.4 Sample Size Determination

The experimental design dictates the execution of 200 to 300 distinct structured prompts, distributed evenly across the six functional categories, four artificial intelligence tools, and two programming ecosystems. Based on pilot methodologies observed in parallel supply chain vulnerability research, this prompt volume is projected to yield between 500 and 1,000 distinct dependency recommendations [Spracklen et al., 2024]. This target sample size is mathematically determined to be sufficient to achieve statistical power ($p < 0.05$) when performing quantitative variance analysis, ensuring that the measured 5 percent baseline threshold for the frequency hypothesis is statistically significant rather than anomalous.

3.4.5 Data Storage and Organization

All generated code, extracted dependency logs, and associated metadata are stored in a secure, version-controlled repository. To maintain strict isolation and prevent the accidental local execution of generated malicious code, the data storage environment is completely decoupled from active execution paths or continuous integration pipelines. The raw text and parsed dependencies are organized into normalized tabular formats (CSV and JSON) to facilitate immediate integration with the automated parsing and validation scripts utilized in the subsequent dependency extraction and risk assessment phases.

3.5 Dependency Extraction and Hallucination Detection

The dependency extraction and hallucination detection phase operationalize the primary dependent variable of the study, defined as the prevalence of vulnerable dependencies. This phase systematically quantifies the absolute count of fabricated application programming interfaces and packages recommended by the artificial intelligence tools across varying configurations of the independent variable. To

maintain a strict quantitative and reproducible posture, the methodology relies on programmatic extraction and validation rather than manual subjective review.

3.5.1 Dependency Extraction Protocol

To achieve high experimental validity and eliminate human transcription errors, the research employs an automated dependency extraction protocol. This protocol utilizes language-specific parsing mechanisms to isolate all external library recommendations embedded within the generated code snippets [Zhao et al., 2025].

3.5.1.1 Spring Boot Dependency Extraction

The Spring Boot testbed represents a compiled, strictly typed ecosystem. The extraction protocol for this environment targets both the project configuration files and the static source code.

pom.xml Parsing

For code generation tasks structured around Maven, automated XML parsers are utilized to scan the generated project object model files. The parsers systematically isolate the Group ID, Artifact ID, and Version tags within the dependencies block to construct a list of required external coordinates.

build.gradle Parsing

For code generation tasks structured around Gradle, the extraction protocol employs regular expression matching to parse the Groovy or Kotlin domain-specific language syntax. The scripts extract the dependency declarations, specifically targeting compile, implementation, and runtimeOnly configurations.

Java Source Code Import Scanning

Beyond the build configuration files, the raw Java source files are processed using static analysis. The protocol extracts the fully qualified class names from all Java import statements, filtering out the standard Java Development Kit core libraries (e.g., java.util, java.io) to isolate only the external third-party recommendations.

3.5.1.2 Node.js Dependency Extraction

The Node.js testbed represents an interpreted, dynamically typed ecosystem. The extraction protocol is adapted to handle the flexible module resolution mechanics inherent to JavaScript development.

package.json Parsing

Automated JSON parsers ingest the generated configuration files, iterating through the dependencies, devDependencies, and peerDependencies key-value pairs to extract the names and requested semantic versioning ranges of all recommended external libraries.

JavaScript Source Code Import Scanning

To identify dependencies invoked dynamically or missing from the package.json file, the raw JavaScript source files are parsed into abstract syntax trees. This allows the extraction algorithm to identify variable assignments and function calls that invoke external modules, regardless of the developer's formatting style.

require() and import() Statement Extraction

The protocol specifically targets the idiomatic module import patterns of the JavaScript language [Zhao et al., 2025]. Regular expression filters and abstract syntax tree node evaluations are programmed to capture both legacy CommonJS require declarations and modern ECMAScript module import statements, ensuring the complete dependency tree is accurately extracted.

3.5.2 Validation Protocol

The core of the positivist measurement strategy relies on programmatically validating the existence of the extracted symbols against authoritative package repositories. This phase determines the physical reality of the moderating variable, registry architecture, by testing whether the generated dependencies actually exist in their respective ecosystems.

3.5.2.1 Maven Central Registry Lookup

For the Spring Boot environment, the extracted Group IDs and Artifact IDs are programmatically queried against the official Maven Central search application programming interface. Because Maven Central enforces a domain-verified namespace architecture, the query confirms whether the generated artifact maps to a registered, cryptographically signed domain [Al-Zofi, 2025].

3.5.2.2 npm Registry Lookup

For the Node.js environment, the extracted package names are queried against the official npm registry application programming interface. In this first-come-first-served flat namespace architecture, the system checks for a standard HTTP 200 OK response to confirm package existence, or an HTTP 404 Not Found response which flags the namespace as currently unregistered and highly susceptible to slopsquatting attacks [Duan et al., 2020].

3.5.2.3 Manual Verification for Ambiguous Cases

While the primary validation is automated, programmatic queries occasionally return non-standard error codes, server timeouts, or partial name matches. In these instances, a human-in-the-loop manual verification step is executed. Researchers manually search the respective registry websites to resolve false positives or negatives caused by application programming interface throttling or transient network errors.

3.5.2.4 Documentation and Repository Presence Verification

To ensure that a package is a true artificial intelligence hallucination rather than a recently deprecated or renamed project, supplementary verification is conducted. The validation protocol cross-references the extracted package names against public GitHub repositories and official developer documentation frameworks to confirm historical existence.

3.5.3 Classification Framework

The final output of the validation phase categorizes each extracted dependency to accurately compute the quantitative count required for the dependent variable.

Valid Dependencies

Dependencies are classified as valid if they return a positive confirmation from the official registry application programming interface and possess verifiable public documentation. These represent successful, non-vulnerable code generation outputs.

Hallucinated Dependencies

Dependencies are classified as hallucinated if they conform to the syntactical standards of the target language but return a definitive non-existent status from the official registry [Spracklen et al., 2025]. These packages represent the primary measure of the attack surface, as they are syntactically plausible but entirely fabricated entities available for malicious registration.

Ambiguous Cases

Dependencies are classified as ambiguous if they represent standard internal language libraries incorrectly formatted as external packages, typographical errors of legitimate packages, or legacy modules removed from active indexes. To maintain statistical conservatism in hypothesis testing, ambiguous cases are excluded from the final vulnerability count.

3.5.4 Dependency Classification Workflow

The complete classification workflow operates as a linear, deterministic pipeline. Initially, the raw code generated across varying workflow types is ingested by the extraction scripts. The language-specific parsers isolate the module names and route them to the validation engine. The validation engine dispatches concurrent application programming interface queries to the appropriate registry architecture (Maven Central or npm). The resulting HTTP response codes dictate the primary classification path. Packages returning definitive responses are immediately categorized as valid or hallucinated. Packages returning undefined or error states are routed to the manual verification and documentation review queue. The pipeline terminates by outputting

the final integer count of confirmed hallucinated dependencies, which serves as the empirical measurement for the prevalence of vulnerable dependencies variable.

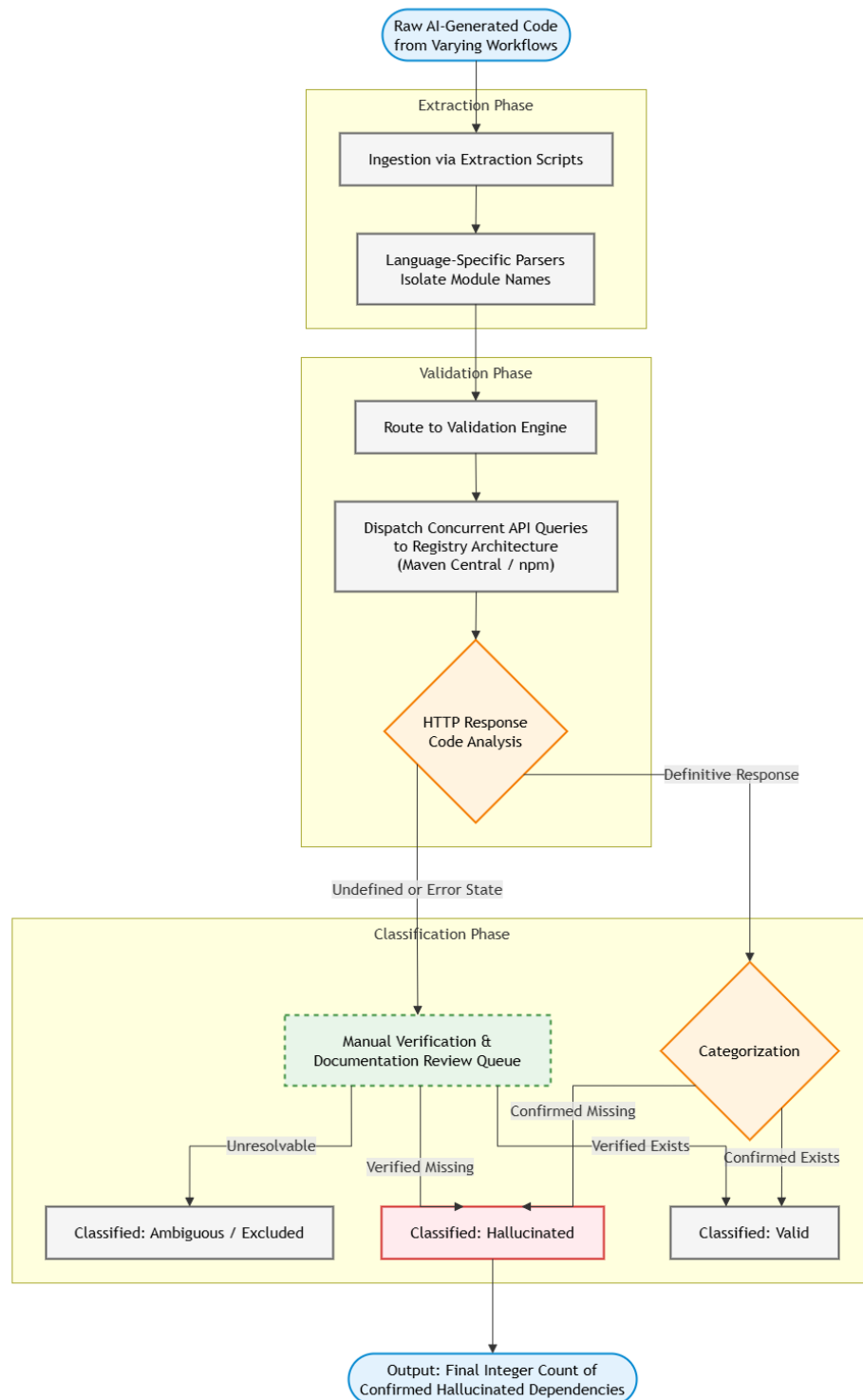


Figure 3-2 : *Dependency Classification Workflow Pipeline*

3.6 Risk Assessment and Exploitability Analysis

Following the extraction and validation of the dependent variable, the research methodology advances to a structured risk assessment to evaluate the precise severity of the identified hallucinated dependencies. Because not all non-existent packages present an equal security threat, this phase operationalizes a quantitative severity scoring framework. The framework mathematically evaluates how the independent and moderating variables interact to dictate the ultimate exploitability of the generated artifacts.

3.6.1 Risk Assessment Dimensions

To construct an objective severity score, the study evaluates each confirmed hallucination across four distinct quantitative and categorical dimensions. This multidimensional approach ensures the risk assessment isolates mechanical vulnerabilities from theoretical errors.

3.6.1.1 Registry Exposure

The registry exposure dimension directly measures the impact of the moderating variable, Registry Architecture, on the viability of a supply chain compromise.

Namespace Availability Assessment

Once a hallucinated dependency is identified, its corresponding namespace is queried to determine its registration status. If the application programming interface returns an explicit "Not Found" error, the namespace is mathematically coded as highly vulnerable, representing an open vector for immediate malicious claiming.

Registration Requirements Analysis

This step evaluates the administrative friction required to register the available namespace. Hallucinations targeting flat architectures, such as the Node Package Manager, are scored with high exposure due to first-come-first-served policies. Conversely, hallucinations targeting domain-verified environments, such as Maven Central, are assigned minimal exposure scores, as attackers cannot feasibly spoof cryptographic domain ownership to fulfill the slopsquatting attack [Al-Zofi, 2025]; [Duan et al., 2020].

3.6.1.2 Naming Similarity Assessment

The second dimension evaluates the lexical plausibility of the generated artifact, assessing whether the hallucination is a slight deviation from a known library or a completely novel invention.

Lexical Similarity to Existing Packages

The methodology employs the Levenshtein distance algorithm to compute the edit distance between the hallucinated package string and the top one thousand most downloaded legitimate packages in the target ecosystem. Hallucinations exhibiting an edit distance of one to three characters are flagged as high-risk variations [Zheng et al., 2026].

Confusion Attack Potential

Based on the lexical distance, the study quantifies the potential for dependency confusion. Hallucinations that share common prefixes or suffixes with highly popular enterprise libraries are weighted more heavily, as these variants bypass natural human skepticism and increase the probability of successful execution [Spracklen et al., 2025].

3.6.1.3 Functional Criticality

This dimension evaluates the operational context in which the artificial intelligence model recommended the fabricated dependency, testing the criticality hypothesis of the study.

Security-Critical Code Paths

Hallucinations injected into security-critical functional categories, such as authentication handlers, cryptography integrations, or database connection pools, are scored with maximum severity. A successful exploit in these code paths grants the attacker immediate access to sensitive data and elevated execution privileges.

Peripheral Functionality

Hallucinated dependencies recommended for peripheral tasks, such as terminal output formatting or benign string manipulation, receive lower severity weights. While still exploitable, the limited scope of the requested functionality typically restricts the immediate blast radius of the compromised package.

3.6.1.4 Cross-Tool Consistency

To evaluate the tool consistency hypothesis, the risk assessment measures how frequently a specific hallucinated package is independently generated across different levels of the independent variable (Workflow Type).

Hallucination Overlap Analysis

The methodology calculates the Jaccard similarity index across the outputs of the tested artificial intelligence models. This quantitative measure determines whether an identical fabricated namespace is generated by multiple distinct foundation models or if it is a localized anomaly.

Attacker Attractiveness Assessment

Packages demonstrating high cross-tool consistency present the highest risk. If multiple tools reliably suggest the same non-existent library, the return on investment for an attacker to claim that specific namespace increases exponentially. These highly consistent hallucinations receive a risk multiplier in the final severity calculation.

3.6.2 Severity Scoring Framework

The outputs from the four assessment dimensions are aggregated into a unified Severity Scoring Framework. This composite score standardizes the dependent variable, enabling robust statistical correlations between the varying degrees of artificial intelligence autonomy and the resulting security posture.

High Severity Classification

A hallucinated dependency is classified as High Severity if it targets a flat namespace architecture, is currently unregistered and available for claiming, and is situated within an autonomous agentic workflow where local execution privileges exist [Gandhi, 2025]. These instances represent zero-day slopsquatting vulnerabilities requiring zero human interaction to trigger a system compromise.

Medium Severity Classification

A dependency is classified as Medium Severity if the namespace is available for registration but occurs within a traditional autocomplete workflow requiring manual developer acceptance. This classification also applies to hallucinations generated in peripheral functional categories where system access is constrained.

Low Severity Classification

Low Severity classifications are assigned to hallucinated packages that target domain-verified architectures, effectively preventing malicious registration [Duan et al., 2020]. Additionally, packages that contain severe syntactical errors preventing compilation, or names that have already been defensively reserved by registry maintainers, are categorized as low risk.

3.6.3 Exploitability Assessment Methodology

The final phase of the methodology translates the calculated severity scores into real-world exploitability metrics, mapping the experimental data directly to the Three-Stage Threat Model defined in the conceptual framework.

Attack Vector Feasibility

This assessment measures the probabilistic likelihood that a threat actor can successfully execute the secondary stage of the threat model. It combines the namespace availability metrics with the attacker attractiveness assessment to quantify how easily and profitably a malicious actor could deploy a slopsquatting payload [Al-Zofi, 2025].

Developer Installation Probability

To assess the final stage of the threat model, the methodology integrates the moderating variable, Developer Expertise. Utilizing data captured from the pre-study survey instruments, the research applies probabilistic weighting to determine the likelihood that developers of varying expertise levels (Novice, Intermediate, Advanced) will manually accept, fail to verify, and ultimately install the hallucinated dependency [Liu et al., 2026].

Potential Damage Assessment

The potential damage is calculated by evaluating the execution environment dictated by the Workflow Type. The methodology distinguishes between the isolated blast radius of a developer's local integrated development environment and the systemic, network-wide impact of an autonomous coding agent operating with read and write permissions across an enterprise filesystem [Qu et al., 2026]. This final assessment confirms the exploitability hypothesis by quantifying the exact percentage of dependencies that translate into actionable, high-impact security breaches.

3.7 Conceptual Framework and Risk Assessment Model

3.7.1 Model Development

3.7.1.1 Integration of Frequency, Exploitability, and Severity

The ultimate objective of this quantitative methodology is to compute a definitive measure of the artificial intelligence hallucination attack surface. To achieve this, the research mathematically integrates three primary dimensions of the dependent variable: Hallucination frequency establishes the baseline volumetric risk, representing the absolute probability that a generative model will recommend a non-existent artifact during a given structural coding task [Spracklen et al., 2024]. Exploitability potential functions as an environmental filter, interacting with the moderating variables to determine what percentage of those baseline hallucinations physically manifest as unregistered, claimable namespaces [Al-Zofi, 2025]. Finally, the severity score provides an impact multiplier, factoring in the functional criticality of the generated code and the execution privileges inherent to the selected workflow type. By calculating the product of these three dimensions, the study transitions from merely counting errors to formally quantifying actionable supply chain threats.

3.7.1.2 Model Architecture

The risk assessment model architecture operationalizes the interactions mapped in the conceptual framework into a sequential computational pipeline. The model initiates at the generation phase, where the independent variables (model architecture, workflow

type, programming environment, functional category, and prompt characteristics) determine the initial hallucination frequency. This raw output is then passed through an environmental filtering phase, where the moderating variable of registry architecture dictates the exploitability potential of the fabricated dependencies. The surviving exploitable artifacts enter the final execution phase, where the moderating variable of developer expertise and the mediating variable of the verification layer apply probabilistic reductions to the attack success rate. The terminal output of this architecture is the finalized prevalence of vulnerable dependencies.

[Insert Figure 3.x: Visual representation of the Risk Assessment Model architecture, mapping the flow from independent variable inputs through moderating filters to the final severity output.]

3.7.2 Model Parameters and Weighting

3.7.2.1 Parameter Definition

The risk assessment model relies on a strict set of parameters directly mapped to the study's operationalization strategy. The primary parameters include the Workflow Type, quantified as a categorical variable distinguishing between reactive autocomplete assistants and autonomous agentic engines [Gandhi, 2025]. The Registry Architecture parameter is defined categorically to contrast first-come-first-served flat namespaces against domain-verified infrastructures [Duan et al., 2020]. The human element is parameterized through Developer Expertise, utilizing an ordinal scale to measure familiarity with supply chain threats. Finally, the defensive parameter is defined by the Verification and Mitigation Layer, recorded as a categorical presence of static or dynamic analysis controls.

3.7.2.2 Weighting Scheme Justification

To ensure the risk model accurately reflects the modern software supply chain threat landscape, the weighting scheme applies deductive logic based on established exploitation requirements. The Workflow Type parameter applies the heaviest impact weight to agentic workflows; because these systems possess direct file-system access and package execution privileges, the requirement for human interaction is bypassed, drastically elevating the severity of a hallucination [Qu et al., 2026]. Similarly, the Registry Architecture parameter assigns maximum exploitability weights to flat

namespaces, as the friction to execute a slopsquatting attack requires zero cryptographic circumvention [Al-Zofi, 2025]. Conversely, the mediating parameter of the Verification Layer applies a reductive weight to the final severity score, mathematically quantifying the interception efficacy of automated security tools prior to code execution.

3.7.3 Model Applicability Across Environments

Spring Boot Environment Adaptation

When the risk assessment model is applied to the Spring Boot and Java ecosystem, the mathematical weighting adapts to the structural realities of a compiled, domain-verified environment. Because Maven Central requires cryptographic verification of namespace ownership, the exploitability potential parameter for external package claiming is heavily reduced [Duan et al., 2020]. Consequently, the model shifts the severity calculation for this ecosystem. While the hallucination frequency may remain statistically significant, the resulting risk profile categorizes these artifacts primarily as functional build failures or localized denial-of-service vectors, rather than highly exploitable external supply chain hijacking opportunities.

Node.js Environment Adaptation

The application of the model to the Node.js and JavaScript ecosystem represents the maximum threat configuration. In this environment, the Registry Architecture parameter dynamically adjusts the model to account for the npm flat namespace policy, which lacks domain verification and operates on a first-come-first-served basis [Duan et al., 2020]. Therefore, any generated dependency that is confirmed as unregistered receives a maximum exploitability weight. The model correctly predicts that within the Node.js environment, the transition from a generative hallucination to an active slopsquatting vulnerability faces minimal structural resistance, rendering the ecosystem highly vulnerable to automated dependency injection attacks [Yadav et al., 2026].

3.8 Mitigation Strategy Formulation

The final phase of the research methodology involves mapping the empirical findings derived from the risk assessment model into actionable defensive protocols. This phase systematically translates the quantitative metrics associated with the prevalence of vulnerable dependencies into targeted interventions. By isolating the effects of the independent variable, moderating variables, and mediating variable, the methodology constructs a precise mitigation framework tailored to distinct points of interception within the software supply chain.

3.8.1 Stakeholder-Specific Recommendations

To address the multifaceted nature of artificial intelligence code hallucinations, the mitigation strategy formulation is structured around four primary stakeholder domains. This operational approach ensures that countermeasures are applied directly to the variables that each stakeholder group can control or influence.

3.8.1.1 Developer-Level Strategies

Developer-level strategies focus on interventions applied at the immediate point of code generation, evaluating the practical application of the mediating variable defined as the Verification and Mitigation Layer.

Automated Dependency Verification Workflows

The methodology proposes the integration of automated dependency verification workflows to function as a dynamic runtime validation mechanism. This strategy tests the efficacy of programmatic queries against authoritative registry application programming interfaces to intercept non-existent packages before local installation commands are executed [AlSobeh et al., 2025]. By measuring the reduction in hallucination acceptance rates when this mediating layer is active, the study quantifies the value of real-time registry validation.

Security Linting Rules

To complement dynamic analysis, the mitigation framework incorporates static analysis interventions. The study evaluates the deployment of strict security linting rules designed to parse the abstract syntax trees of generated code snippets [Tileria et

al., 2026]. This strategy assesses whether static verification can successfully flag unverified package coordinates or anomalous application programming interface calls before the code progresses to the compilation or build phase.

IDE Integration Recommendations

Because the workflow type dictates the level of artificial intelligence autonomy, the methodology evaluates mitigation strategies embedded directly within the integrated development environment. The framework assesses the technical requirement for integrated development environments to enforce hard interruptions when autonomous agentic workflows attempt to execute shell commands involving unverified external libraries [Yadav et al., 2026].

3.8.1.2 Organizational-Level Strategies

Organizational-level strategies are designed to manipulate the moderating variable of developer expertise and establish standardized operating procedures for utilizing artificial intelligence coding assistants.

AI Tool Usage Policies

The methodology evaluates the necessity of strict usage policies that govern which workflow types are permissible within enterprise environments. By analyzing the differential exploitability rates between traditional autocomplete tools and autonomous agentic engines, the study provides an empirical basis for organizations to restrict highly autonomous tools from operating in environments with write privileges [Qu et al., 2026].

Developer Training Guidelines

To positively shift the developer expertise variable, the mitigation framework outlines specific training directives. The study measures how targeted education regarding software supply chain threats and the deterministic failure modes of large language models impacts the baseline vulnerability acceptance rate [Gandhi, 2025]. This approach formally links higher ordinal scores in developer security awareness to a measurable decrease in the successful installation of hallucinated dependencies.

Code Review Procedures

The framework integrates mandatory human-in-the-loop code review procedures as an organizational countermeasure. The methodology assesses the statistical reduction in exploitability when strict peer-review protocols are enforced, specifically focusing on the manual verification of all external package imports introduced by artificial intelligence suggestions [Dubey and Madiseti, 2026].

3.8.1.3 Tool Vendor Recommendations

Tool vendor recommendations target the underlying architecture of the generative models to reduce the baseline hallucination frequency before outputs reach the end user.

Training Data Curation

The methodology proposes data curation strategies aimed at reducing the generation of deprecated, hypothetical, or contextually invalid libraries. By analyzing the cross-tool consistency of fabricated packages, the framework highlights the necessity for vendors to filter training corpora against active registry indexes to prevent models from learning and reproducing phantom dependencies [Spracklen et al., 2025].

Post-Generation Validation

The framework evaluates the implementation of post-generation validation layers directly within the model inference pipeline. This strategy tests the theoretical application of knowledge graph reasoning and retrieval-augmented generation to force models to cross-reference their structural outputs against verified software documentation prior to rendering the response [Jain et al., 2025]; [Li et al., 2026].

3.8.1.4 Registry Maintainer Recommendations

Registry maintainer recommendations directly address the moderating variable of registry architecture, evaluating infrastructural changes required to neutralize the slopsquatting attack vector.

Proactive Namespace Monitoring

The mitigation framework proposes proactive namespace monitoring for first-come-first-served package repositories. The methodology assesses the feasibility of utilizing

predictive generation to identify commonly hallucinated package names and preemptively reserving or blocking these high-risk namespaces from public registration [Al-Zofi, 2025].

Malicious Registration Prevention

To address the fundamental architectural disparities evaluated in the study, the framework outlines recommendations for transitioning flat registries toward domain-verified namespace models. By demonstrating the protective capacity of environments like Maven Central, the methodology provides quantitative justification for enforcing cryptographic domain ownership and strict provenance checks to eliminate the attack surface for fabricated packages [Duan et al., 2020].

3.8.2 Feasibility Assessment

To ensure the proposed mitigation strategies possess practical utility, the methodology concludes with a formal feasibility assessment. This phase measures the operational viability of implementing the defined mediating variables within active software development lifecycles.

Implementation Complexity

The assessment quantifies the technical complexity required to deploy the recommended verification layers. This includes evaluating the integration overhead of connecting static syntax parsers and dynamic registry validation application programming interfaces into existing continuous integration pipelines and local integrated development environments.

Workflow Disruption Analysis

Because artificial intelligence coding tools are primarily adopted to increase developer velocity, the methodology measures the workflow disruption introduced by the mitigation strategies. The analysis calculates the latency added by real-time dependency validation and the frequency of false-positive interruptions, determining if the friction introduced by the security controls negates the productivity benefits of the autonomous workflow [Liu et al., 2026].

Cost-Benefit Considerations

The final methodological step is a quantitative cost-benefit analysis. This procedure correlates the measured reduction in the prevalence of vulnerable dependencies against the computational and operational costs of maintaining the verification and mitigation layer. This calculation ensures that the recommended defensive posture is both statistically effective and economically justifiable for enterprise deployment.

3.9 Model Validation

The validation phase establishes the empirical rigor and generalizability of the proposed risk assessment model. Because the dependent variable involves the complex intersection of generative failures and real-world supply chain vulnerabilities, the methodology must systematically prove that the calculated severity scores accurately represent actionable threats. This validation is conducted through a positivist, tripartite approach consisting of internal statistical validation, external generalization testing, and parametric sensitivity analysis.

3.9.1 Internal Validation

Internal validation ensures that the mathematical architecture of the risk assessment model is structurally sound and that the classification of hallucinated dependencies is highly reproducible without subjective bias.

3.9.1.1 Dataset Split Strategy

To rigorously test the predictive parameters of the risk model, the aggregated dataset of extracted code dependencies is subjected to a randomized partitioning process. This split prevents the model configurations from overfitting to specific anomalies present in a singular batch of generated code.

Training Set (70%)

Seventy percent of the collected dependency data is allocated to the training set. This primary subset is utilized to calibrate the weighting schemes of the moderating variables, specifically determining how registry architecture policies and functional code criticality numerically influence the baseline hallucination frequency to produce a stable severity score.

Testing Set (30%)

The remaining thirty percent of the data is strictly isolated as the testing set. This holdout data is processed through the finalized risk assessment model to objectively measure the framework's ability to consistently assign severity classifications to previously unseen package artifacts without further parametric tuning.

3.9.1.2 Predictive Accuracy Assessment

The predictive accuracy assessment quantifies the model's precision in identifying immediately exploitable attack vectors. The experimental outputs from the testing set are cross-referenced against live registry queries. The assessment measures the ratio of dependencies the model flagged as high severity against the actual availability of those targeted namespaces in the corresponding ecosystem. A high predictive accuracy confirms the exploitability hypothesis, proving the model correctly isolates theoretical syntactical errors from actionable slopsquatting targets.

3.9.1.3 Inter-Rater Reliability Testing

To maintain absolute quantitative rigor, the dependency validation protocol requires verification that the classification of ambiguous artifacts is not influenced by researcher bias.

Independent Security Expert Classification

A subset of the extracted dependencies, specifically those categorized as ambiguous or requiring manual documentation verification, is distributed to a panel of independent security experts [Gao et al., 2025]. These reviewers independently apply the classification framework criteria to determine whether the suggested package constitutes a valid library, a typographical error, or a true hallucination.

Cohen's Kappa Coefficient

To statistically quantify the level of consensus among the independent reviewers, the methodology calculates the Cohen's Kappa coefficient [Zhuo et al., 2025]. This calculation measures inter-rater agreement while accounting for the possibility of random chance. A Kappa score approaching 1.0 indicates near-perfect agreement, validating that the operationalized definitions of hallucinated dependencies are objective, reproducible, and robust for experimental measurement.

3.9.2 External Validation

External validation tests the boundaries of the research design, ensuring that the identified vulnerabilities are not isolated artifacts of the specific time period or the primary artificial intelligence tools utilized during initial data collection.

3.9.2.1 Cross-Temporal Validation

The software supply chain is highly dynamic; package registries frequently update their namespace policies and deprecate legacy modules.

3.9.2.1.1 Temporal Holdout Sets

To account for temporal shifts in registry architectures, a secondary data collection phase is executed after a predetermined chronological interval. This temporal holdout set generates a new batch of code snippets utilizing the same standardized prompts [Pashchenko et al., 2022].

Generalization Assessment

The generalization assessment compares the hallucination frequency and exploitability potential of the temporal holdout set against the primary dataset. Consistent statistical findings across both time periods validate that artificial intelligence dependency fabrication is a persistent architectural flaw of large language models rather than a temporary anomaly in their training data.

3.9.2.2 Cross-Tool Validation

To verify the tool consistency hypothesis and confirm that the extraction protocols are universally applicable, the methodology incorporates cross-tool testing.

Alternative AI Tool Testing

The standardized prompt repository is submitted to alternative artificial intelligence coding models that were excluded from the primary independent variable testing [Twist et al., 2026]. This includes exposing the prompts to differing foundational architectures, contrasting the outputs of open-weight community models against the previously tested commercial proprietary systems.

Model Robustness Evaluation

The robustness evaluation measures whether the automated dependency extraction parsers successfully identify and isolate external library calls generated by these alternative tools. A successful cross-tool validation demonstrates that the extraction instrumentation is resilient to diverse syntactical formatting and idiosyncratic model outputs.

3.9.2.3 Expert Review

While the methodology maintains a strict quantitative posture, structural validation is supplemented through structured expert review to confirm the real-world applicability of the severity classifications.

Industry Security Professional Review

The calculated severity scores and the resulting threat models are reviewed by active software supply chain security practitioners. This review ensures that the definitions of exploitability align with contemporary adversarial capabilities, specifically validating the assumption that flat namespace architectures present an immediate, high-severity risk.

Academic Researcher Validation

The operationalization table and the structural logic of the mediating variables are subjected to peer review by academic methodologists. This validation confirms that the experimental test metrics accurately capture the constructs defined in the conceptual framework.

3.9.3 Sensitivity Analysis

The final validation step involves a rigorous mathematical sensitivity analysis. Because the severity scoring framework relies on assigned numerical weights for different risk dimensions, it is necessary to prove that the core experimental findings are not artificially manufactured by arbitrary parametric values.

Risk Dimension Weighting Variations

The sensitivity analysis systematically manipulates the mathematical weights assigned to the moderating variables, such as Developer Expertise and Registry Architecture.

The algorithm iteratively recalculates the final severity scores for the entire dataset under varying weighting distributions.

Threshold Impact Analysis

The methodology adjusts the numerical boundaries that separate low, medium, and high severity classifications. This analysis measures how shifting these thresholds impacts the total count of critical vulnerabilities, determining whether the frequency of actionable exploits remains statistically significant under highly conservative threshold constraints.

Robustness Assessment

The outcome of the sensitivity analysis is a definitive robustness assessment. If the data continuously demonstrates that agentic workflow types operating in flat registry architectures produce a high prevalence of vulnerable dependencies regardless of the specific weighting variations applied, the model is deemed highly robust. This confirms the validity of the positivist experimental design and solidifies the accuracy of the tested hypotheses.

3.10 Data Structure and Documentation

3.10.1 Dataset Structure

To ensure the rigorous quantitative measurement required by a positivist research design, the collected data is structured into a normalized, relational database. This format systematically links the independent variable permutations to their corresponding dependent variable outcomes. The dataset is organized into distinct field categories, ensuring that every phase of the code generation and validation process is captured for statistical analysis.

Prompt Information Fields

This category records the initial conditions of the experiment. Fields include a unique prompt identifier, the exact natural language string submitted to the model, and the functional category being tested (e.g., database connectivity, authentication). These fields ensure that input variance is strictly controlled and reproducible.

AI Tool Data Fields

These fields capture the independent variable configuration for each trial. The recorded data includes the specific workflow type utilized (traditional autocomplete versus agentic coding workflow), the foundation model architecture queried, and the exact software version or application programming interface endpoint accessed during the test run [Gandhi, 2025].

Generated Code Fields

This segment stores the raw textual output produced by the artificial intelligence tools. Fields include the complete synthesized code snippet, the designated programming language, and any syntactical constraints or formatting parameters explicitly requested in the prompt. Storing the raw output allows for retrospective analysis of the code semantics beyond the immediate dependency extraction.

Dependency Data Fields

These fields document the extracted artifacts isolated by the automated parsing scripts. The data includes the parsed array of required package names, application programming interfaces, namespace coordinates (such as Group ID and Artifact ID for Java), and explicitly declared version numbers.

Validation Results Fields

This category captures the empirical measurement of the primary dependent variable. Fields record the specific registry application programming interface queried, the exact HTTP response code returned, the classification of the dependency (valid, hallucinated, or ambiguous), and the binary outcome indicating whether the mediating verification and mitigation layer successfully flagged the anomaly [Li et al., 2026].

Risk Indicators Fields

The final dataset segment maps the moderating variables to the exploitability outcomes. Fields include the registry architecture type (first-come-first-served versus domain-verified), the computed lexical similarity scores to existing packages, the namespace availability status, and the final aggregated severity score.

3.10.2 Documentation Standards

Metadata Recording

Comprehensive metadata is recorded alongside every generated dataset entry to preserve experimental context. This includes documenting the system execution environment, the temperature and token-limit parameters configured for the artificial intelligence application programming interfaces, and the specific version states of the automated extraction parsers.

Timestamp Documentation

Because the software supply chain is inherently dynamic, the exact chronological execution of the experiment is a critical variable. Timestamps are documented for both the moment of code generation and the exact moment of registry validation. This precision ensures that subsequent shifts in registry state such as a legitimate package being deprecated or a hallucinated name being maliciously registered by an external actor do not invalidate the empirical findings recorded at the time of testing [Pashchenko et al., 2022].

Version Control

All data collection scripts, parsing algorithms, and validation utilities are maintained within a secure version control system. The raw extracted data files (stored as normalized JSON and CSV formats) are committed to immutable storage repositories, allowing peer researchers to inspect the data evolution and verify the integrity of the transformation processes applied during the risk assessment phase.

Reproducibility Measures

To align with scientific transparency standards, the methodology ensures full reproducibility by strictly defining the prompt templates, seed values (where applicable), and execution architectures. By standardizing the input conditions and utilizing automated parsing instrumentation, external researchers can replicate the experimental testbed to verify the reported prevalence of vulnerable dependencies across alternative models or future software iterations [Zhao et al., 2025].

3.11 Ethical Considerations

3.11.1 Live Registry Manipulation Avoidance

A critical ethical constraint in this methodology is the absolute avoidance of live registry manipulation. While the study evaluates the exploitability of hallucinated packages, the research design strictly prohibits the active registration of the fabricated namespaces identified during the experiment. Claiming these packages, even for benign proof-of-concept testing, actively introduces unverified artifacts into the open-source ecosystem, creating unnecessary noise for registry maintainers and potentially disrupting legitimate development pipelines [Al-Zofi, 2025]. The exploitability potential is instead measured safely and quantitatively by querying the registry application programming interfaces for availability status (e.g., HTTP 404 Not Found) without executing the final claiming phase.

3.11.2 Responsible Disclosure Considerations

In the event that the data collection phase reveals a systemic, previously undocumented vulnerability pattern within a specific artificial intelligence model, or uncovers a live slopsquatting attack actively exploiting a hallucinated dependency, the research adheres to responsible disclosure protocols. Findings presenting an immediate, high-severity threat to enterprise supply chains will be embargoed and securely disclosed to the respective tool vendors or registry administrators prior to academic publication.

3.11.3 Research Integrity and Transparency

The study maintains a strict positivist posture, ensuring that data is reported without selective omission. All code generation attempts, including those resulting in compilation failures or benign outcomes, are included in the final statistical analysis to prevent the artificial inflation of the hallucination frequency metric.

3.11.4 Data Privacy and Security

The methodology integrates a moderating variable concerning developer expertise, which relies on survey data collected from human participants. To comply with ethical data protection standards, all responses regarding professional experience and security familiarity are thoroughly anonymized. No personally identifiable information is linked

to the composite expertise scores, and the aggregated demographic data is stored on encrypted, access-controlled institutional servers.

3.12 Methodological Limitations

3.12.1 Scope Constraints

Exclusion of Mobile and Desktop Development

To maintain controlled experimental parameters, the scope of this study is strictly constrained to backend server and web application environments. Mobile application frameworks (such as Android SDK or iOS Swift) and compiled desktop environments are excluded from the testbed. These ecosystems rely on fundamentally different build architectures, proprietary app store validation policies, and heavily sandboxed dependency resolution mechanisms, which introduce external variables that cannot be consistently compared against standard open-source package managers.

Transitive Dependencies Exclusion

The methodology calculates the prevalence of vulnerable dependencies based exclusively on the direct, top-level libraries explicitly recommended in the generated code snippets. The study does not execute deep dependency tree resolution to identify secondary or transitive hallucinations. While malicious actors frequently exploit deep transitive layers, limiting the measurement to direct imports isolates the generative failure directly attributable to the artificial intelligence model, removing noise introduced by the complex resolution logic of the underlying package manager [Duan et al., 2020].

3.12.2 Temporal Snapshot Limitation

The findings produced by this quantitative design represent a strict temporal snapshot. Both the foundation models evaluated and the software registries queried are continuously evolving systems. Model providers frequently deploy silent weights updates, and open-source ecosystems index thousands of new packages daily. Consequently, the specific hallucination frequencies and exploitability metrics calculated during this study are valid exclusively for the specific timeframe in which

the registry application programming interfaces were queried, and these rates may fluctuate as models undergo subsequent alignment training [Pashchenko et al., 2022].

3.12.3 Sample Size Considerations

While the prompt volume (200 to 300 queries) is statistically powered to evaluate the predefined functional categories and detect the baseline 5 percent hallucination threshold, the sample size cannot exhaustively represent the entirety of all possible software engineering tasks. The constraints of application programming interface rate limits and processing latency mean that highly obscure, long-tail programming languages or niche legacy frameworks are not represented in the dataset. Therefore, the generalization of the resulting severity scores is most accurate when applied to mainstream, high-velocity development ecosystems.

4. References

[Agarwal et. al., 2024] Agarwal, V., Pei, Y., Alamir, S. and Liu, X., CodeMirage: Hallucinations in code generated by large language models, <https://doi.org/10.48550/arXiv.2408.08333>, 2024.

[Al-Zofi, 2025] Al-Zofi, W., AI-Induced Supply-Chain Compromise: A Systematic Review of Package Hallucinations and Slopsquatting Attacks, *Research Square*, <https://www.researchsquare.com/article/rs-8007192/v1>, 2025.

[Daoud, 2026] Daoud, S., Security-related code hallucinations in Large Language Models: A Systematic Literature Review, in *Proceedings of the 2026 International Symposium of Systems, Advanced Technologies and Knowledge (ISSATK)*, Hammamet, Tunisia, <https://ieeexplore.ieee.org/document/11566869>, 2026.

[Duan et. al., 2020] Duan, R., Alrawi, O., Kasturi, R. P., Elder, R., Saltaformaggio, B. and Lee, W., Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages, in *Proceedings of the Network and Distributed System Security Symposium (NDSS 2020)*, San Diego, USA, <https://ui.adsabs.harvard.edu/abs/2020arXiv200201139D>, 2020.

[Dubey and Madiseti, 2026] Dubey, S. and Madiseti, V. K., eXAI-CodeDebug Towards Reliable, Trustworthy, and Explainable LLM-Based Coding for Production Systems, *Computing in Science & Engineering*, vol. 28, no. 2, pp. 1 – 9, <https://doi.org/10.1109/MCSE.2026.3684488>, 2026.

[Gandhi, 2026] Gandhi, D., Survey On Systemic Security Risks in Vibe Coding and Autonomous Development Workflows, <https://doi.org/10.36227/techrxiv.176800890.09196406/v1>, 2026.

[Gao et. al., 2025] Gao, C., Fan, G., Chong, C. Y., Chen, S., Liu, C., Lo, D., Zheng, Z. and Liao, Q., A Systematic Literature Review of Code Hallucinations in LLMs: Characterization, Mitigation Methods, Challenges, and Future Directions for Reliable AI, *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, pp. 1–45, <https://doi.org/10.48550/arXiv.2511.00776>, 2025.

[Jain et. al., 2025] Jain, N., Kwiatkowski, R., Ray, B., Ramanathan, M. K. and Kumar, V., On Mitigating Code LLM Hallucinations with API Documentation, in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Ottawa, Canada, <https://ieeexplore.ieee.org/abstract/document/11121691>, 2025.

[Zhao et. al., 2025] Zhao, Y., Wu, M., Hu, X. and Xia, X., HFuzzer: Testing Large Language Models for Package Hallucinations, in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, <https://doi.org/10.1109/ASE63991.2025.00225>, 2025.

[AlSobeh et. al., 2025] AlSobeh, A., Gwarzo, Z. and Shatnawi, A., ShadowPlay: Engineering Defenses Against Role-Based Prompt Injection and Dependency Hallucination in LLM-Powered Development, in *IEEE*, <https://doi.org/10.1109/Cyber-AI66431.2025.11233258>, 2025.

[Ladisa et. al., 2023] Ladisa, P., Plate, H., Martinez, M. and Barais, O., SoK: Taxonomy of Attacks on Open-Source Software Supply Chains, in *Proceedings of the 2023 IEEE European Symposium on Security and Privacy (EuroS&P)*, Delft, Netherlands, pp. 150–167, <https://ieeexplore.ieee.org/abstract/document/10179304>, 2023.

[Le-Anh et. al., 2026] Le-Anh, M., Nguyen, H., Tran, A. K., Hai, N. L., Van, L. N., Bui, N. D. Q. and Le, B., Do Not Treat Code as Natural Language: Implications for Repository-Level Code Generation and Beyond, *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, pp. 120–142, <https://dl.acm.org/doi/10.1145/3797144>, 2026.

[Li et. al., 2026] Li, Y., Ma, Z., Zou, Y., Wang, Y., Yang, L. and Xie, B., Progressively Mitigating API Hallucination in LLM-Generated Code via Knowledge Graph Reasoning, in *Proceedings of the 2026 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Rovaniemi, Finland, <https://ieeexplore.ieee.org/document/11576711/>, 2026.

[Lian et. al., 2024] Lian, X., Wang, S., Ma, J., Tan, X., Liu, F. and Shi, L., Imperfect Code Generation: Uncovering Weaknesses in Automatic Code Generation by Large Language Models, in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion (ICSE-Companion)*, Lisbon,

Portugal, pp. 240–242, <https://dl.acm.org/doi/10.1145/3639478.3643081>, 2024.

[Liu, 2026] Liu, X., Alleviating Code Hallucination in Large Language Models via Pseudocode Intermediation, in *Proceedings of the 2026 9th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, Jinan, China, <https://ieeexplore.ieee.org/document/11509160/>, 2026.

[Liu et. al., 2025a] Liu, C., Lin, H. Y. and Thongtanunam, P., Hallucinations in Code Change to Natural Language Generation: Prevalence and Evaluation of Detection Metrics, in *Proceedings of the 14th International Joint Conference on Natural Language Processing (IJCNLP-AACL 2025)*, Mumbai, India, pp. 2538–2560, <https://doi.org/10.18653/v1/2025.ijcnlp-long.137>, 2025.

[Liu et. al., 2025b] Liu, J., Zhang, Y., Wang, D., Li, Y. and Dong, W., THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge, in *Proceedings of the 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Montreal, QC, Canada, <https://ieeexplore.ieee.org/document/10992555/>, 2025.

[Liu et. al., 2026] Liu, F., Liu, Y., Shi, L., Yang, Z., Zhang, L., Lian, X., Li, Z. and Ma, Y., Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code, *IEEE Transactions on Software Engineering*, vol. 52, no. 3, pp. 1037 - 1055, <https://ieeexplore.ieee.org/document/11361549/>, 2026.

[Ohm and Stuke, 2023] Ohm, M. and Stuke, C., SoK: Practical Detection of Software Supply Chain Attacks, in *Proceedings of the 18th International Conference on Availability, Reliability and Security (ARES)*, Gubbio, Italy, pp. 1–11, <https://dl.acm.org/doi/10.1145/3600160.3600162>, 2023.

[Pashchenko et. al., 2022] Pashchenko, I., Vu, D. and Massacci, F., Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies, *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592-1609, <https://ieeexplore.ieee.org/abstract/document/9201023>, 2022.

[Qu et. al., 2026] Qu, Y., Liu, Y., Geng, T., Deng, G., Li, Y., Zhang, L., Zhang, Y. and Ma, L., Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill

Ecosystems, <https://doi.org/10.48550/arXiv.2604.03081>, 2026.

[Spracklen et. al., 2025] Spracklen, J., Wijewickrama, R., Sakib, A.H.M.N., Maiti, A., Viswanath, B. and Jadliwala, M., We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs, in Proceedings of the 34th USENIX Conference on Security Symposium, <https://dl.acm.org/doi/10.5555/3766078.3766268>, 2025.

[Tian et. al., 2025] Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L. and Song, D., CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification, <https://doi.org/10.1609/aaai.v39i24.34717>, 2025.

[Tileria et. al., 2026] Tileria, M., Dash, S. K., Pârțachi, P. P. and Barr, E. T., Hallucination Inspector: A Fact-Checking Judge for API Migration, <https://doi.org/10.48550/arXiv.2604.20202>, 2026.

[Tripathi et. al., 2025] Tripathi, Y., Kohli, V. and Misra, S., Confident but Incorrect: Mitigating Hallucination and Overconfidence in Agentic AI Coders, in *Proceedings of the 2025 Eighth International Conference on Image Information Processing (ICIIP)*, Solan, India, <https://ieeexplore.ieee.org/document/11346318/>, 2025.

[Twist et. al., 2026] Twist, L., Zhang, J. M., Harman, M., Syme, D., Noppen, J., Yannakoudakis, H. and Nauck, D., Library Hallucinations in LLM-Generated Code: A Risk Analysis Grounded in Developer Queries, <https://doi.org/10.48550/arXiv.2509.22202>, 2026.

[Wang et. al., 2025] Wang, S., Ouyang, L., Zhang, J., Yu, T., Wang, X. and Sun, J., Large Language Model Supply Chain: A Research Agenda, *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, pp. 1-32, <https://dl.acm.org/doi/full/10.1145/3708531>, 2025.

[Washio and Miyao, 2022] Washio, K. and Miyao, Y., Code Generation for Unknown Libraries via Reading API Documentations, <https://doi.org/10.48550/arXiv.2202.07806>, 2022.

[Williams et. al., 2025] Williams, L., Benedetti, G., Hamer, S., Paramitha, R., Rahman, I., Tamanna, M., Tystahl, G., Zahan, N., Morrison, P., Acar, Y., Cukier, M.,

Kästner, C., Kapravelos, A., Wermke, D. and Enck, W., Research Directions in Software Supply Chain Security, *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1-38, <https://dl.acm.org/doi/full/10.1145/3714464>, 2025.

[Woesle et. al., 2025] Woesle, C., Fischer-Brandies, L. and Buettner, R., A Systematic Literature Review of Hallucinations in Large Language Models, *IEEE Access*, vol. 13, pp. 12450-12472, <https://ieeexplore.ieee.org/document/11132297>, 2025.

[Yadav et. al., 2026] Yadav, H., Singh, V., Sharma, K., Karmarkar, S. and Agarwal, S., AI-Assisted Software Development as a New Supply-Chain Attack Surface, *Journal of Pioneering Artificial Intelligence Research*, vol. 2, no. 2, pp. 1-8, doi.org/10.63721/26JPAIR0129, 2026.

[Yang et. al., 2026] Yang, B., Dang, J. and Liu, H., Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection, in *Proceedings of the 2026 IEEE International Conference on Software Reliability*, pp. 412-421, <https://ieeexplore.ieee.org/document/11278592/>, 2026.

[Zheng et. al., 2026] Zheng, J., Guan, H. and Liu, Y., When LLMs Invent Rust Crates: An Empirical Study of Hallucination Patterns and Mitigation, in *Proceedings of the 17th International Conference on Internetware*, <https://doi.org/10.48550/arXiv.2606.08444>, 2026.

[Zhuo et. al., 2025] Zhuo, T. Y., He, J., Sun, J., Xing, Z., Lo, D., Grundy, J. and Du, X., Identifying and Mitigating API Misuse in Large Language Models, <https://doi.org/10.1109/TSE.2026.3651566>, 2025.